

Laborator 1

1 Obiective

Obiectivul acestui laborator este de a descrie pe scurt biblioteca SDL și de a exemplifica dezvoltarea unei aplicații bazate pe aceasta.

2 Crearea unei ferestre folosind SDL

Biblioteca SDL (Simple DirectMedia Layer) gestionează accesul la hardware-ul grafic (prin biblioteci OpenGL și Direct3D), precum și audio, tastatură și mouse (independent de sistemul de operare folosit). Această bibliotecă suportă mai multe sisteme de operare precum Windows, Mac OS X și Linux. Pe parcursul acestui laborator vom folosi limbajul de programare C++ pentru dezvoltarea aplicațiilor, dar există implementări SDL disponibile și pentru alte limbaje (cum ar fi C # sau Python).

Pentru a crea o fereastră SDL sunt necesari următorii pași:

1. Inițializați biblioteca SDL apelând funcția [SDL_Init\(\)](#) cu argumentul [SDL_INIT_VIDEO](#), deoarece folosim numai subsistemul video al SDL.

```
SDL_Init(SDL_INIT_VIDEO);
```

2. Creați o fereastră utilizând funcția [SDL_CreateWindow\(\)](#). Argumentele sunt titlul ferestrei, poziția, lățimea, înălțimea și unele flag-uri (reprezentând opțiuni - de exemplu, pentru a crea o fereastră pe tot ecranul sau o fereastră redimensionabilă).

```
SDL_CreateWindow("SDL Hello World Example", SDL_WINDOWPOS_UNDEFINED,  
SDL_WINDOWPOS_UNDEFINED, WINDOW_WIDTH, WINDOW_HEIGHT, SDL_WINDOW_SHOWN |  
SDL_WINDOW_ALLOW_HIGHDPI);
```

Înainte de a închide aplicația, trebuie să dealocăm toate resursele create:

1. DISTRUGEȚI fereastra apelând funcția [SDL_DestroyWindow\(\)](#) și oferind ca argument pointerul către fereastră.

```
SDL_DestroyWindow(window);
```

2. Apelați funcția [SDL_Quit\(\)](#) care este responsabilă pentru curățarea tuturor subsistemelor inițializate (pentru exemplul nostru folosim doar subsistemul video).

```
SDL_Quit();
```

3 SDL Surface

Pentru a desena ceva pe ecran avem nevoie de un obiect “canvas” (pânză). În SDL, un “canvas” poate fi reprezentat de **SDL_Surface** sau **SDL_Texture**. **SDL_Surface** este folosit în redarea de tip software, în schimb **SDL_Texture** este utilizat în redarea de tip hardware, diferența principală fiind localizarea buffer-elor de date (pixeli). Pe parcursul acestui laborator vom folosi **SDL_Surface**, deoarece vom încerca să implementăm pipeline-ul grafic de la zero.

În SDL, o suprafață este o structură de date care conține o colecție de pixeli. Fiecare fereastră are atașată o astfel de suprafață și, în plus, putem crea noi suprafețe și aplica unele operații pe acestea (de exemplu, putem încărca o imagine pe o suprafață și copia conținutul acesteia în fereastră). [SDL Surface](#) stochează formatul pixelilor, dimensiunea (lățimea și înălțimea), un pointer la datele efective ale pixelilor și alte informații relevante. Vom folosi o reprezentare a pixelilor pe 32 de biți. În această situație stocăm 1 octet per canal (roșu, verde, albastru și alfa).

Dacă dorim să desenăm un dreptunghi, trebuie să specificăm poziția de pornire a dreptunghiului (coordonatele x și y ale colțului stânga-sus) și dimensiunea (lățimea și înălțimea). Pentru a specifica o culoare folosim funcția **SDL_MapRGB()** utilizând formatul pixel al suprafeței și valorile de culoare roșu, verde și albastru. Funcția **SDL_FillRect()** va actualiza suprafața dreptunghiului folosind culoarea specificată.

```
SDL_Rect rectangleCoordinates = {100, 100, 200, 200};  
Uint32 rectagleColor = SDL_MapRGB(windowSurface->format, 255, 0, 0);  
SDL_FillRect(windowSurface, &rectangleCoordinates, rectagleColor);
```

Pentru fiecare canal de culoare specificăm valorile cuprinse între 0 și 255. În tabelul următor se regăsesc câteva exemple de culori care se pot obține folosind diferite combinații de valori ale celor trei canale.

Nume	Canal Roșu	Canal Verde	Canal Albastru	Culoare
Alb	255	255	255	
Roșu	255	0	0	
Verde	0	255	0	
Albastru	0	0	255	
Galben	255	255	0	
Negru	0	0	0	

4 Procesarea evenimentelor

În SDL evenimentele ar putea fi lucruri de genul apăsării unei taste de tastatură sau mișcarea mouse-ului. Toate evenimentele sunt stocate într-o coadă în ordinea în care au apărut. Folosind funcția [SDL_WaitEvent\(\)](#) primim următorul eveniment din coadă.

4.1 Evenimente de apăsare a mouse-ului

Putem verifica dacă butonul stânga al mouse-ului a fost apăsat folosind următorul fragment de cod. Mai întâi verificăm ca tipul de eveniment să fie **SDL_MOUSEBUTTONDOWN** și apoi ca butonul apăsat să fie **SDL_BUTTON_LEFT**. Funcția **SDL_GetMouseState()** primește coordonatele poziției curente a mouse-ului (x și y).

```
if(currentEvent.type == SDL_MOUSEBUTTONDOWN)
{
    if(currentEvent.button.button == SDL_BUTTON_LEFT)
    {
        SDL_GetMouseState(&mouseX, &mouseY);
    }
}
```

4.2 Evenimente de mișcare a mouse-ului

Putem verifica dacă butonul stânga al mouse-ului este apăsat în timp ce mouse-ul se mișcă utilizând următorul fragment de cod. Mai întâi verificăm ca tipul evenimentului să fie **SDL_MOUSEMOTION** și apoi ca butonul apăsat pentru să fie **SDL_BUTTON_LEFT**.

```
if(currentEvent.type == SDL_MOUSEMOTION)
{
    if(currentEvent.button.button == SDL_BUTTON_LEFT)
    {
        SDL_GetMouseState(&mouseX, &mouseY);
    }
}
```

4.3 Evenimente generate de tastatură

Putem obține tasta apăsată utilizând următorul fragment de cod. Mai întâi verificăm ca tipul de eveniment să fie **SDL_KEYDOWN** și apoi procesăm tastele dorite.

```
if(currentEvent.type == SDL_KEYDOWN)
{
    switch(currentEvent.key.keysym.sym)
    {
        case SDLK_UP:
            //process UP key
            break;

        case SDLK_R:
            //process R key
            break;

        ...

        default:
```

```
//default process  
  
break;  
  
}
```

5 Lectură suplimentară

- Instalarea SDL pe diferite platforme – http://lazyfoo.net/SDL_tutorials/lesson01/index.php
- Tutoriale SDL - http://lazyfoo.net/SDL_tutorials/

6 Temă

Descărcați și rulați aplicația de pe site-ul Web al laboratorului. Încercați să înțelegeți exemplul de bază și apoi extindeți aplicația cu următoarele funcționalități:

- Schimbați culoarea dreptunghiului prin setarea canalelor de culoare folosind tastele R (pentru roșu), G (pentru verde) și B (pentru albastru) și modificând valoarea canalului prin apăsarea tastelor UP și DOWN.
- Afișați interactiv dreptunghiul. Primul set de coordonate (colțul stanga-sus) este preluat la evenimentul de apăsare al butonului stânga al mouse-ului și al doilea set de valori (lățimea și înălțimea dreptunghiului) este calculat la fiecare eveniment de mișcare a mouse-ului (dar numai dacă butonul din stânga este încă apăsat – desenare dreptunghi cu Drag and Drop, similară celei din utilitarul Microsoft Paint).

Laborator 2

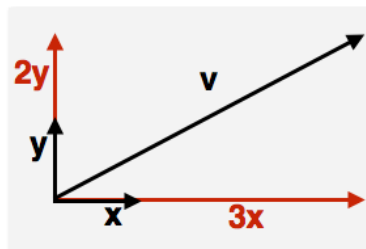
1 Obiective

Obiectivul acestui laborator este de a implementa un set de clase C++ pentru manipularea vectorilor.

2 Vectori

În domeniul graficii computerizate se utilizează vectori pentru a determina unghiul dintre muchii, orientarea suprafețelor, poziția relativă a unui punct pe o suprafață, calculul modelelor de iluminare și multe alte lucruri.

Vectorii se reprezintă printr-o listă de numere și grafic într-un sistem de coordonate cartezian. Reprezentăm vectorii ca săgeți și le numim folosind litere cu font **bold**. Din punct de vedere geometric, un vector este descris prin direcție și lungime. În 2D, un vector poate fi scris ca o combinație de doi vectori care nu sunt paraleli (cu o lungime diferită de 0). Un vector $\mathbf{v}$ este reprezentat de $\mathbf{v} = v_x \mathbf{x} + v_y \mathbf{y}$ unde v_x, v_y sunt coordonatele carteziane ale vectorului. Putem să scriem vectori pe orizontală și îi numim vectori rând sau îi putem scrie vertical și îi vom numi vectori coloană. Pentru exemplul anterior putem scrie: $\mathbf{v} = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$, or $\mathbf{v}^T = [v_x \ v_y]$.



În figura precedentă vectorul $\mathbf{v}$ poate fi definit ca o combinație a vectorilor de bază $\mathbf{x}$ și $\mathbf{y}$.

$$\mathbf{v} = 3\mathbf{x} + 2\mathbf{y} = 3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

În domeniul graficii computerizate suntem interesați de vectori 2D, 3D și 4D și ne referim la elementele acestora folosind:

- x, y (în 2D)
- x, y, z (în 3D)
- x, y, z, w (în 4D)

Vectorii pot fi utilizați pentru a stoca deplasarea (decalajul dintre două puncte) sau locații (reprezentate ca deplasare dintr-o origine bine cunoscută). Rețineți însă că locațiile nu sunt vectori (nu putem adăuga "Cluj" la "București").

Diferența dintre două puncte este un vector ($\mathbf{v} = Q - P$), iar suma dintre un punct și un vector este un punct ($\mathbf{v} + P = Q$).

3 Operații

3.1 Modulul unui vector

Lungimea (modulul) este notată cu $\|\mathbf{v}\|$ și este egală cu rădăcina pătrată a sumei pătratului elementelor vectoriale. În cazul 2D, lungimea este $\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2}$.

Un **vector unitate** este un vector cu lungimea egală cu 1. **Vectorul zero** are lungimea egală cu 0 (în acest caz direcția este nedefinită).

3.2 Normalizarea unui vector

Putem **normaliza** orice vector non-zero împărțind vectorul la lungimea sa. Noul vector este orientat în aceeași direcție cu vectorul original, dar lungimea lui este egală cu 1.

$$\mathbf{v} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

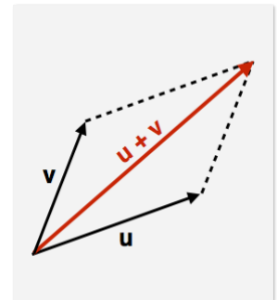
3.3 Adunarea vectorilor

Putem aduna doi vectori însumând elementele corespunzătoare din vectorii inițiali. Dacă avem doi vectori 2D $\mathbf{u} = \begin{bmatrix} u_x \\ u_y \end{bmatrix}$ și $\mathbf{v} = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$ rezultatul operației de adunare este un vector egal cu:

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_x + v_x \\ u_y + v_y \end{bmatrix}$$

Operația de adunare a vectorilor este comutativă:

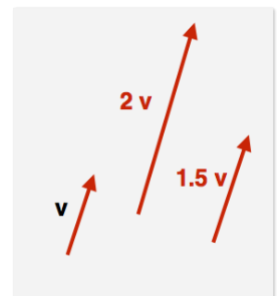
$$\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$$



3.4 Scalarea vectorilor

Putem scala un vector cu o valoare scalară pentru a schimba lungimea vectorului. Pentru aceasta, înmulțim elementele vectorului cu valoarea scalară. Dacă scalăm un vector cu -1 schimbăm direcția vectorului.

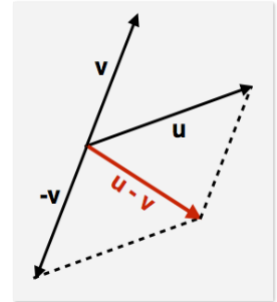
$$\beta \mathbf{u} = \begin{bmatrix} \beta u_x \\ \beta u_y \end{bmatrix}$$



3.5 Scăderea vectorilor

Scăderea a doi vectori este similară cu operația de adăugare, scalând cu -1 vectorul secund.

$$\mathbf{u} - \mathbf{v} = \mathbf{u} + (-\mathbf{v}) = \begin{bmatrix} u_x - v_x \\ u_y - v_y \end{bmatrix}$$



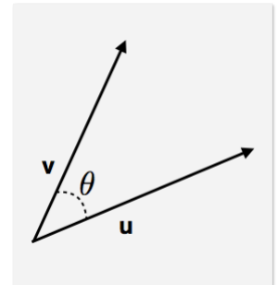
3.6 Înmulțirea vectorilor

3.6.1 Produsul scalar

Produsul scalar (dot product) dintre doi vectori returnează o valoare scalară dependentă de lungimea vectorilor și de unghiul dintre ei.

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

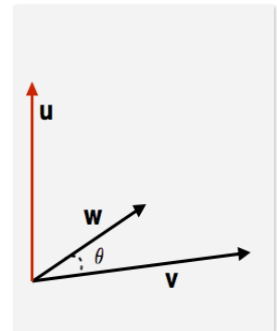
Dacă doi vectori $\mathbf{u}$ and $\mathbf{v}$ sunt reprezentați în coordonate Carteziane atunci $\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y$. Similar, în 3D, produsul scalar este: $\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y + u_z v_z$.



3.6.2 Produsul vectorial

În domeniul graficii computerizate folosim în principal acest produs numai pe vectori 3D, dar produsul poate fi generalizat. Rezultatul este un vector perpendicular pe cei doi vectori. Modulul vectorului rezultat este $\|\mathbf{v} \times \mathbf{w}\| = \|\mathbf{v}\| \|\mathbf{w}\| \sin \theta$. Direcția vectorului este perpendiculară pe planul format de cei doi vectori inițiali iar sensul se poate determina folosind regula mâinii drepte sau regula șurubului.

$$\mathbf{v} \times \mathbf{w} = \begin{bmatrix} v_y w_z - v_z w_y \\ v_z w_x - v_x w_z \\ v_x w_y - v_y w_x \end{bmatrix}$$



4 Temă

Descărcați codul sursă de pe pagina web a laboratorului. Trebuie să implementați metodele în interiorul fișierelor sursă (vec2.cpp, vec3.cpp și vec4.cpp). Fișierele antet (header) conțin definiția claselor și metodele care ar trebui implementate.

Laborator 3

1 Obiective

Obiectivul acestui laborator este implementarea unor clase C++ specifice pentru operațiile cu matrici.

2 Matricea

O matrice este o colecție de numere aranjate în rânduri și coloane. În contextul graficii computerizate folosim conceptul de matrice pentru a reprezenta transformările spațiale. Numărul de rânduri și coloane este egal în acest caz, indicând o matrice pătratică. În acest laborator folosim matrici 3x3 și 4x4.

3 Matricea identitate

Matricea identitate este matricea care are 1 pe diagonală și 0 în celelalte poziții. De exemplu, matricea identitate 3x3 este:

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4 Operații

4.1 Înmulțirea cu un scalar

Prin multiplicarea matricei $\mathbf{M}$ cu un scalar k se înmulțește fiecare element al matricei cu valoarea scalară.

$$k\mathbf{M} = k \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} = \begin{bmatrix} km_{11} & km_{12} & km_{13} \\ km_{21} & km_{22} & km_{23} \\ km_{31} & km_{32} & km_{33} \end{bmatrix}$$

4.2 Adunarea

Adunarea matricelor se face element cu element, ca în acest exemplu:

$$\mathbf{M} + \mathbf{N} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} + \begin{bmatrix} n_{11} & n_{12} & n_{13} \\ n_{21} & n_{22} & n_{23} \\ n_{31} & n_{32} & n_{33} \end{bmatrix} = \begin{bmatrix} m_{11} + n_{11} & m_{12} + n_{12} & m_{13} + n_{13} \\ m_{21} + n_{21} & m_{22} + n_{22} & m_{23} + n_{23} \\ m_{31} + n_{31} & m_{32} + n_{32} & m_{33} + n_{33} \end{bmatrix}$$

4.3 Înmulțirea cu o matrice

Pentru înmulțirea a două matrici, numărul de coloane din prima matrice trebuie să fie același cu numărul de rânduri din cea de-a doua matrice.

$$\begin{bmatrix} a_{11} & \dots & a_{1m} \\ \vdots & & \vdots \\ a_{i1} & \dots & a_{im} \\ \vdots & & \vdots \\ a_{r1} & \dots & a_{rm} \end{bmatrix} \begin{bmatrix} b_{11} & \dots & b_{1j} & \dots & b_{1c} \\ \vdots & & \vdots & & \vdots \\ b_{m1} & \dots & b_{mj} & \dots & b_{mc} \end{bmatrix} = \begin{bmatrix} p_{11} & \dots & p_{1j} & \dots & p_{1c} \\ \vdots & & \vdots & & \vdots \\ p_{i1} & \dots & p_{ij} & \dots & p_{ic} \\ \vdots & & \vdots & & \vdots \\ p_{r1} & \dots & p_{rj} & \dots & p_{rc} \end{bmatrix}$$

Elementul p_{ij} este egal cu:

$$p_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{im}b_{mj}$$

Înmulțirea a două matrici nu este comutativă ($\mathbf{M}_1\mathbf{M}_2 \neq \mathbf{M}_2\mathbf{M}_1$), dar este asociativă și distributivă:

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$$

$$(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$$

4.4 Înmulțirea cu un vector coloană

Această operație este un caz special al operației de înmulțire a două matrici.

$$\begin{bmatrix} a_{11} & \dots & a_{1m} \\ \vdots & & \vdots \\ a_{i1} & \dots & a_{im} \\ \vdots & & \vdots \\ a_{r1} & \dots & a_{rm} \end{bmatrix} \begin{bmatrix} b_{11} \\ \vdots \\ b_{j1} \\ \vdots \\ b_{m1} \end{bmatrix} = \begin{bmatrix} p_{11} \\ \vdots \\ p_{i1} \\ \vdots \\ p_{r1} \end{bmatrix}$$

Elementul p_{i1} este egal cu:

$$p_{i1} = a_{i1}b_{11} + a_{i2}b_{21} + \dots + a_{im}b_{m1}$$

4.5 Transpusa

Transpusa unei matrici (notată $\mathbf{M}^T$) este matricea în care se schimbă coloanele și rândurile. De exemplu, pentru o matrice de 3x3:

$$\begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}^T = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

Transpusa produsului dintre două matrici este $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$.

4.6 Determinantul

Pentru o matrice 2x2 determinantul este:

$$|\mathbf{M}| = \begin{vmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{vmatrix} = m_{11}m_{22} - m_{12}m_{21}$$

Pentru a calcula determinantul, trebuie să găsim cofactorii elementelor din matrice. Cofactorul fiecărui element al matricei pătratice este determinantul unei matrici (obținută prin înlăturarea din matricea originală a rândului și coloanei în care se află elementul) înmulțit cu minus unu în unele cazuri. Semnul cofactorului poate fi determinat după următorul model:

$$\begin{bmatrix} + & - & + & \dots \\ - & + & - & \dots \\ + & - & + & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Pentru o matrice 4x4:

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix}$$

Cofactorii pentru primul rând sunt:

$$\begin{aligned} m_{11}^c &= \begin{vmatrix} m_{22} & m_{23} & m_{24} \\ m_{32} & m_{33} & m_{34} \\ m_{42} & m_{43} & m_{44} \end{vmatrix} & m_{12}^c &= - \begin{vmatrix} m_{21} & m_{23} & m_{24} \\ m_{31} & m_{33} & m_{34} \\ m_{41} & m_{43} & m_{44} \end{vmatrix} \\ m_{13}^c &= \begin{vmatrix} m_{21} & m_{22} & m_{24} \\ m_{31} & m_{32} & m_{34} \\ m_{41} & m_{42} & m_{44} \end{vmatrix} & m_{14}^c &= - \begin{vmatrix} m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \\ m_{41} & m_{42} & m_{43} \end{vmatrix} \end{aligned}$$

Determinantul este egal cu suma produselor dintre elemente (de pe orice rând sau coloană) și cofactorii lor.

Pentru o matrice 3x3 determinantul este:

$$|\mathbf{M}| = \begin{vmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{vmatrix} = m_{11} \begin{vmatrix} m_{22} & m_{23} \\ m_{32} & m_{33} \end{vmatrix} - m_{12} \begin{vmatrix} m_{21} & m_{23} \\ m_{31} & m_{33} \end{vmatrix} + m_{13} \begin{vmatrix} m_{21} & m_{22} \\ m_{31} & m_{32} \end{vmatrix}$$

Similar, pentru o matrice 4x4, determinantul este:

$$\begin{aligned} |\mathbf{M}| &= \begin{vmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{vmatrix} = m_{11} \begin{vmatrix} m_{22} & m_{23} & m_{24} \\ m_{32} & m_{33} & m_{34} \\ m_{42} & m_{43} & m_{44} \end{vmatrix} - m_{12} \begin{vmatrix} m_{21} & m_{23} & m_{24} \\ m_{31} & m_{33} & m_{34} \\ m_{41} & m_{43} & m_{44} \end{vmatrix} + \\ & m_{13} \begin{vmatrix} m_{21} & m_{22} & m_{24} \\ m_{31} & m_{32} & m_{34} \\ m_{41} & m_{42} & m_{44} \end{vmatrix} - m_{14} \begin{vmatrix} m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \\ m_{41} & m_{42} & m_{43} \end{vmatrix} \end{aligned}$$

4.7 Inversa unei matrici

Matricea inversă (notată $\mathbf{A}^{-1}$) este maricea pentru care $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Inversa produsului dintre două matrici este $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.

Pentru o matrice 4x4 inversa este:

$$\mathbf{M}^{-1} = \frac{1}{|\mathbf{M}|} \begin{bmatrix} m_{11}^c & m_{21}^c & m_{31}^c & m_{41}^c \\ m_{12}^c & m_{22}^c & m_{32}^c & m_{42}^c \\ m_{13}^c & m_{23}^c & m_{33}^c & m_{43}^c \\ m_{14}^c & m_{24}^c & m_{34}^c & m_{44}^c \end{bmatrix}$$

5 Temă

Descărcați codul sursă de pe pagina web a laboratorului. Trebuie să implementați metodele din interiorul fișierelor sursă (mat3.cpp și mat4.cpp). Fișierele antet conțin definiția claselor și metodele care ar trebui implementate.

Laborator 4

1 Obiective

Acest laborator prezintă noțiunile cheie privind transformările 2D și 3D (translație, scalare, rotație).

2 Definirea punctelor 2D și 3D

Un punct 2D este definit într-un sistem de coordonate omogen prin $(x*w, y*w, w)$. Pentru simplitate în sistemele bidimensionale, parametrul w este setat la 1. Prin urmare, definiția punctului este $(x, y, 1)$.

O altă reprezentare a punctului este următoarea: $P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$.

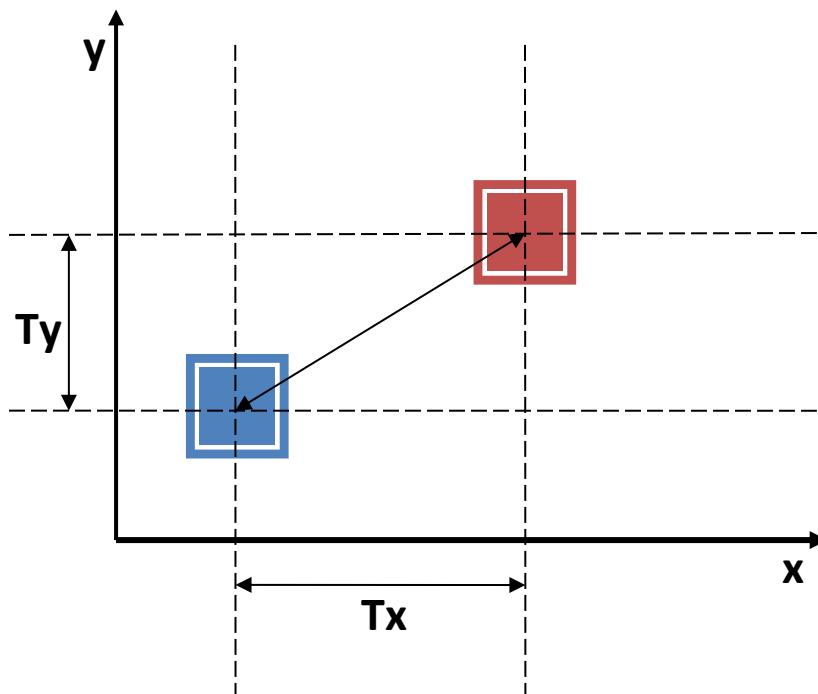
Similar, în 3D definim punctele ca $(x*w, y*w, z*w, w)$ și le reprezentăm ca vectori coloană:

$$P = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3 Transformări

3.1 Translație 2D

Transformarea de translație este folosită pentru a muta un obiect (punct) cu o anumită cantitate.



Matricea operației de translație este următoarea:

$$T = \begin{bmatrix} 1 & 0 & Tx \\ 0 & 1 & Ty \\ 0 & 0 & 1 \end{bmatrix}$$

unde T_x și T_y reprezintă factorii de translație pe axe x și y . Dacă aplicăm transformarea punctului 2D, $P' = T * P$, obținem noi coordonate pentru acel punct:

$$x' = x + Tx$$

$$y' = y + Ty$$

Matricea pentru transformarea inversă este următoarea:

$$T = \begin{bmatrix} 1 & 0 & -Tx \\ 0 & 1 & -Ty \\ 0 & 0 & 1 \end{bmatrix}$$

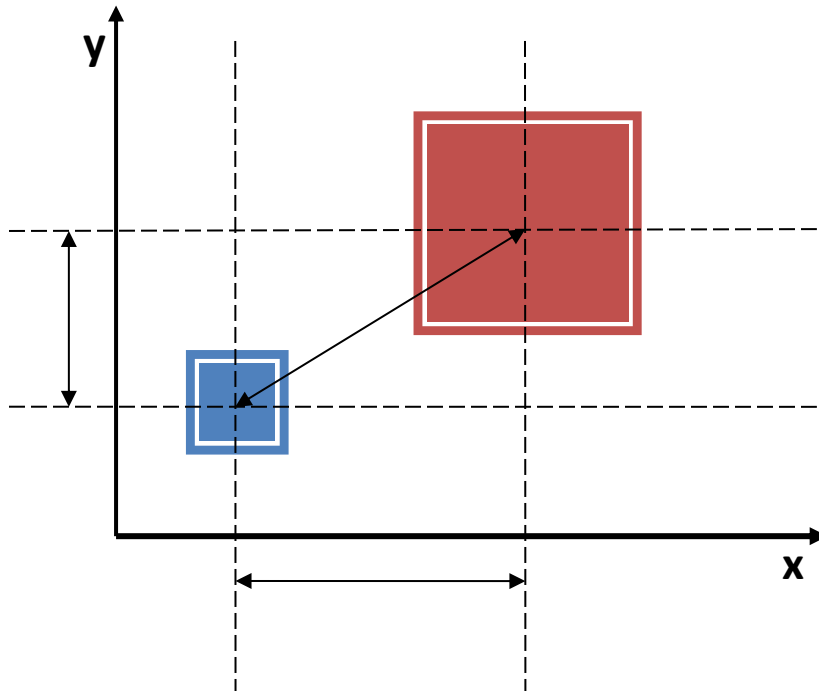
3.2 Translație 3D

Singura diferență față de cazul 2D este că aici avem încă o coordonată și matricea de transformare va fi 4x4.

$$T = \begin{bmatrix} 1 & 0 & 0 & Tx \\ 0 & 1 & 0 & Ty \\ 0 & 0 & 1 & Tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.3 Scalare 2D

Transformarea de scalare mărește sau micșorează un obiect. Transformarea este *relativă la origine*.



Matricea operației de scalare este următoarea:

$$S = \begin{bmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

unde Sx și Sy reprezintă factorii de scalare pe axele x și y . Dacă factorii Sx și Sy sunt egali atunci transformarea de scalare este uniformă. Dacă factorii Sx și Sy nu sunt egali, transformarea de scalare este neuniformă. Dacă aplicăm transformarea la punctul 2D, $P' = S * P$, obținem noi coordonate pentru acel punct:

$$x' = x * Sx$$

$$y' = y * Sy$$

Dacă setăm factorii de scalare la ± 1 , putem reflecta forma originală.

Matricea transformării inverse este următoarea:

$$S = \begin{bmatrix} 1/Sx & 0 & 0 \\ 0 & 1/Sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

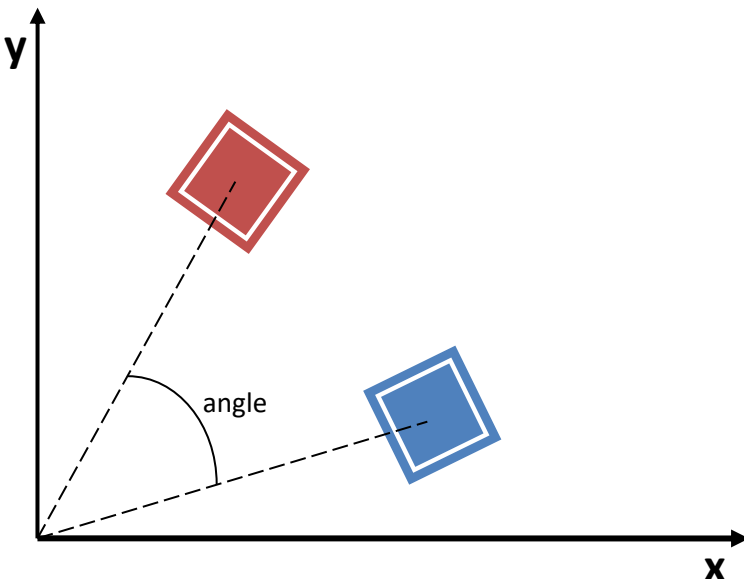
3.4 Scalare 3D

Matricea operației de scalare 3D este următoarea:

$$S = \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.5 Rotație 2D

Această transformare rotește un obiect cu un unghi dat. Această transformare este, de asemenea, *relativă la origine*.



Matricea pentru operația de rotație este următoarea:

$$R = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

unde α reprezintă unghiul de rotație. Dacă aplicăm transformarea punctului 2D, $P' = P * R$, obținem noi coordonate pentru acel punct:

$$x' = x * \cos \alpha - y * \sin \alpha$$

$$y' = x * \sin \alpha + y * \cos \alpha$$

Matricea transformării inverse este următoarea:

$$R = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3.6 Rotație 3D

Specificăm rotația în spațiul 3D independent pe axele x, y și z. Rotația în jurul axei z este similară cu rotația din 2D (coordonatele z rămân neschimbate).

$$R_z = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4 Temă

Descărcați codul sursă de pe pagina laboratorului. Trebuie să implementați metodele din interiorul fișierului sursă (transform.cpp). Fișierul header (transform.h) conține definiția metodelor care ar trebui implementate.

Laborator 7

1 Obiective

Obiectivul acestui laborator este de a folosi operațiile cu vectori și matrici definite în laboratoarele anterioare pentru a efectua diferite transformări pe o figură 2D.

2 Utilizarea aplicației SDL

Pentru exemplificarea grafică a transformărilor 2D, vom folosi o aplicație foarte asemănătoare cu cea din laboratorul 1, bazată pe biblioteca SDL (Simple DirectMedia Layer).

3 SDL_Renderer

SDL_Renderer este o structură de date care gestionează procesul de rasterizare. Aceasta este legată de un **SDL_Window**, astfel încât rasterizarea se poate face doar în acel **SDL_Window**. De asemenea, un obiect de tip **SDL_Renderer** poate să țină evidența setărilor de rasterizare.

Pentru a crea un renderer legat de fereastra aplicației, vom folosi următorul cod:

```
SDL_Renderer *windowRenderer;  
  
windowRenderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);
```

Există câteva funcții importante asociate obiectelor de tip **SDL_Renderer**:

1. **SDL_SetRenderDrawColor** stabilește culoarea care va fi utilizată în toate operațiile de rasterizare, până când se efectuează un alt apel la funcție.

```
SDL_SetRenderDrawColor(renderer, r, g, b, a);
```

2. **SDL_RenderClear** șterge întreaga zonă de afișare a ferestrei utilizând culoarea activă curentă, setată anterior folosind funcția **SDL_SetRenderDrawColor**.

```
SDL_RenderClear(renderer);
```

3. **SDL_RenderPresent** va afișa întregul conținut rasterizat pe suprafața ecranului. Până când această funcție este apelată, procesul de rasterizare are loc într-un buffer ascuns care nu este vizibil pentru utilizator. Apelul **SDL_RenderPresent** trebuie efectuat o singură dată, după ce toate funcțiile de rasterizare au fost apelate.

```
SDL_RenderPresent(renderer);
```


4 Trasarea unei linii

Trasarea unei linii utilizând SDL Renderer necesită următorii pași:

1. Definiți culoarea care urmează să fie folosită (exemplu: albastru):

```
SDL_SetRenderDrawColor(renderer, 0, 0, 255, 255);
```

2. Definiți punctele de început și de sfârșit ale liniei. Pentru aceasta vom folosi variabilele vec3:

```
vec3 P1(100, 100, 1), P2(400, 100, 1);
```

3. Desenați linia folosind obiectul renderer:

```
SDL_RenderDrawLine(windowRenderer, P1.x, P1.y, P2.x, P2.y);
```

4. Afișați conținutul obiectului renderer pe ecran:

```
SDL_RenderPresent(renderer);
```

5 Temă

Descărcați și rulați aplicația de pe site-ul web al laboratorului. Încercați să înțelegeți exemplul de bază și apoi extindeți aplicația cu următoarele funcționalități:

- Includeți în aplicație fișierele de implementare (.cpp).
- Definiți un dreptunghi initial având colțul din stânga-sus în $P_1(100, 100)$ și colțul din dreapta jos în $P_2(400, 200)$.
- Rotiți dreptunghiul în jurul centrului său (intersecția diagonalelor) cu 10 grade în sensul acelor de ceasornic atunci când este apăsat `RIGHT_ARROW` și 10 grade în sens invers acelor de ceasornic atunci când este apăsat `LEFT_ARROW`.
- Scalați dreptunghiul având colțul din stânga sus ca referință, utilizând tastele `UP_ARROW` și `DOWN_ARROW`.

Laborator 8

1 Obiective

Acest laborator prezintă principiile de funcționare ale algoritmilor de decupare a primitivelor grafice. Doi algoritmi pentru decuparea liniilor sunt prezentați: Cyrus-Beck și Cohen-Sutherland.

2 Algoritmi pentru decuparea liniilor

Algoritmii de decupare sunt folosiți pentru a elimina valorile din afara unui anumit domeniu, adică părți ale segmentelor sau poligoanelor care se află în afara zonei de afișare. În majoritatea cazurilor, zona de afișare este definită ca un dreptunghi și se numește fereastră de decupare. În raport cu această fereastră de decupare, o primitivă (punct, linie sau poligon) poate fi în una dintre următoarele relații:

- În întregime în interiorul ferestrei de decupare - nu decupăm primitiva, se continuă afișarea.
- În întregime în exteriorul ferestrei de decupare - nu decupăm primitiva, se renunță la aceasta.
- Intersectează fereastra de tăiere – se calculează punctele de intersecție și se actualizează primitiva, continuă afișarea.

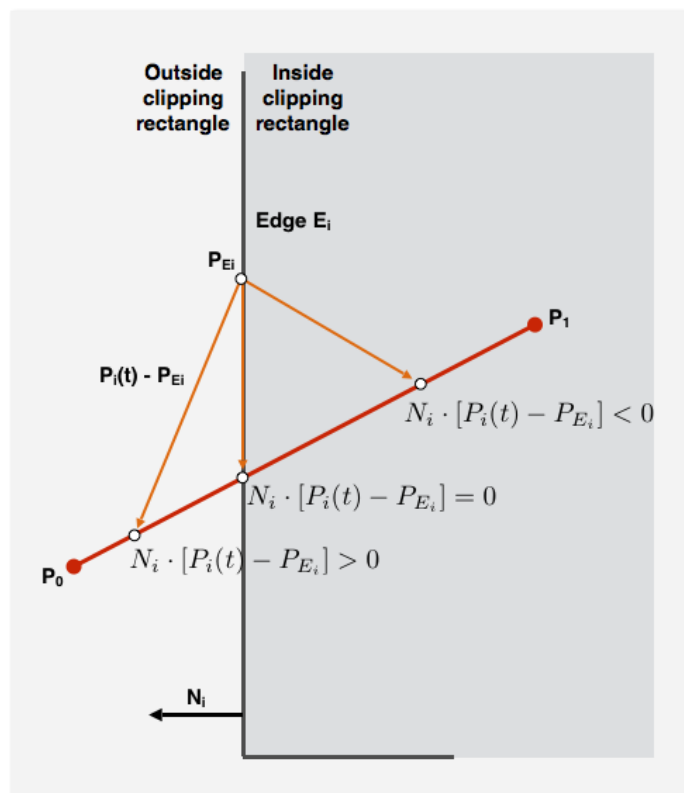
3 Algoritmul de decupare Cyrus-Beck

Algoritmul Cyrus-Beck este folosit pentru a decupa un segment de dreaptă după conturul unui poligon convex. Segmentul de dreaptă este definit parametric sub forma:

$$P(t) = P_0 + (P_1 - P_0)t, \text{ where } t \in [0, 1]$$

Pentru fiecare muchie E_i calculăm vectorul normală astfel încât acesta să fie orientat înspre exterior (precum în figura alăturată). Pentru fiecare muchie se alege un punct P_{Ei} . Calculând produsul scalar $N_i \cdot (P(t) - P_{Ei})$ putem identifica poziția relativă a fiecărui punct din segmentul de dreaptă (definit pentru o anumită valoare a parametrului t):

- Dacă valoarea este **negativă**, punctul se află în semiplanul interior
- Dacă valoarea este **pozitivă**, punctul se află în semiplanul exterior
- Dacă valoarea este **zero**, punctul se află pe muchia ferestrei de decupare



Suntem interesați de valorile lui t pentru care $N_i \cdot (P(t) - P_{Ei}) = 0$.

Mai întâi înlocuim $P(t)$ și alegem un factor comun:

$$N_i \cdot (P_0 + (P_1 - P_0)t - P_{Ei}) = 0$$

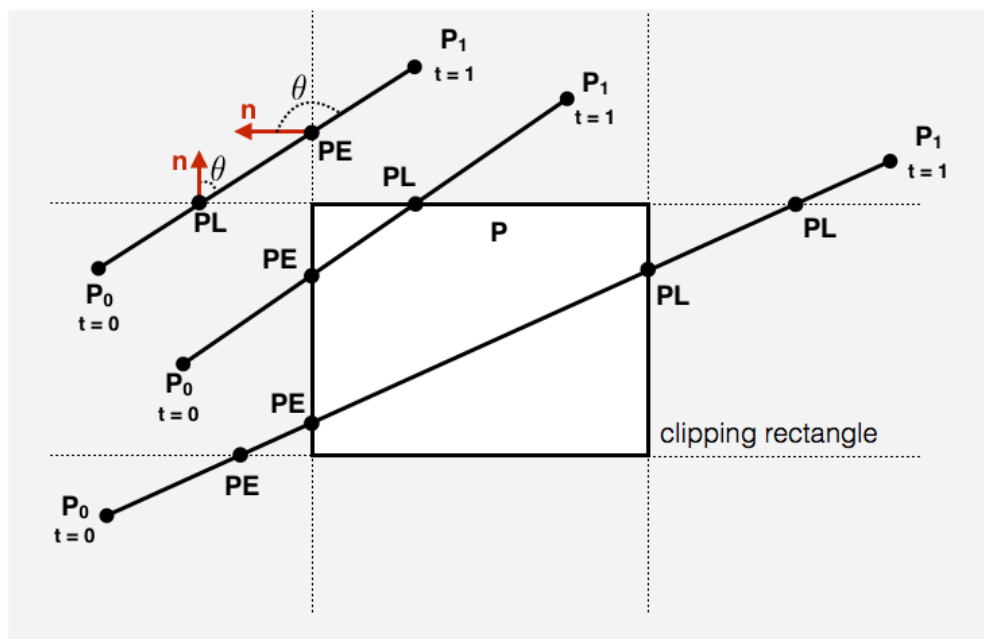
$$N_i \cdot (P_0 - P_{Ei}) + N_i \cdot (P_1 - P_0)t = 0$$

Fie $D = P_1 - P_0$ vectorul definit de punctele P_0 și P_1 . În acest caz, putem calcula t ca:

$$t = \frac{N_i \cdot (P_0 - P_{Ei})}{-N_i \cdot D}$$

Trebuie să calculăm valoarea lui t (a punctului de intersecție) pentru fiecare muchie a poligonului de decupare. Valorile lui t din afara intervalului $[0, 1]$ sunt ignorate. Fiecare intersecție este caracterizată prin calculul unghiului dintre P_0P_1 și N_i ca:

- “potențial intrare” (PE) – dacă unghiul $> 90^\circ$
- “potențial ieșire” (PL) – dacă unghiul $< 90^\circ$



3.1 Pseudocod

```

precalculeaza  $N_i$  și selectează un  $PE_i$  pentru fiecare muchie;
if ( $P_1 = P_0$ )
    linia degenerază într-un punct, deci facem decuparea unui punct;
else
     $t_E = 0$ ;  $t_L = 1$ ;
    
```

```

for (fiecare candidat calculeaza intresectia cu o muchie de decupare){
    if (Ni * D != 0){
        calculeaza t;

        foloseste semnul expresiei Ni * D pentru a categoriza punctul ca PE (potential intrare) sau PL (potential iesire);

        if (PE)
            tE = max(tE, t);

        if (PL)
            tL = min(tL, t);
    }
}

if (tE > tL)
    return -1
else
    return P(tE ) si P(tL ) ca intersectii de decupare

```

4 Algoritmul de decupare Cohen-Sutherland

4.1 Determină dacă un punct P(x,y) este vizibil

Considerând P1(x_{min} , y_{min}), și P2(x_{max} , y_{max}) punctele care definesc dreptunghiul zonei vizibile, punctul P(x,y) este vizibil numai dacă sunt îndeplinite următoarele condiții:

$$x_{min} \leq x \leq x_{max}$$

$$y_{min} \leq y \leq y_{max}$$

4.2 Determină dacă un segment este vizibil

Pentru a determina dacă un segment de dreaptă este vizibil, avem nevoie de algoritmi de o complexitate puțin mai ridicată. O idee ar fi să testăm vizibilitatea fiecărui punct al segmentului, înainte de a fi afișat pe ecran. Dar această metodă va necesita mult timp și foarte multe calcule. Metoda poate fi ușor îmbunătățită prin testarea mai întâi a capetelor segmentului. Dacă ambele puncte sunt în zona vizibilă, întregul segment va fi vizibil. Acest caz se numește "acceptare simplă". Pe aceeași logică, dacă ambele puncte sunt în afara și pe aceeași parte a zonei vizibile, nici o parte a segmentului nu va fi vizibilă. Acest caz este numit "respingere simplă". În toate celelalte cazuri, trebuie să folosim alți algoritmi pentru a stabili care parte a segmentului (dacă există) este vizibilă.

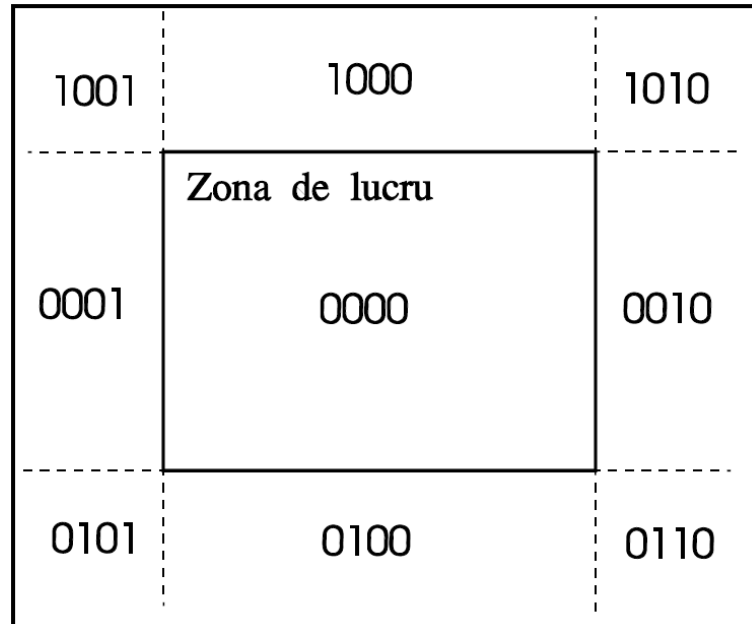
Dacă un segment de linie nu poate fi inclus în cazuri de "acceptare simplă" sau "respingere simplă", atunci trebuie să calculăm punctele sale de intersecție cu următoarele drepte:

$$y = y_{max}, x = x_{max}, y = y_{min}, x = x_{min}$$

și să eliminăm segmentele care sunt plasate în afara zonei vizibile. Ca rezultat, vom obține un nou segment de linie. Algoritmul se repetă până când segmentul rezultat poate fi inclus într-unul din cazurile de "acceptare simplă" sau "respingere simplă".

Algoritmul Cohen-Sutherland folosește un cod de patru cifre pentru a descrie fiecare capăt de segment. Codul are următoarea structură:

- prima cifră este 1 dacă punctul este deasupra zonei vizibile; altfel este 0
- a doua cifră este 1 dacă punctul este sub zona vizibilă; altfel este 0
- a treia cifră este 1 dacă punctul este în partea dreaptă a zonei vizibile; altfel este 0
- a patra cifră este 1 dacă punctul este în partea stângă a zonei vizibile; altfel este 0



4.3 Pseudocod

```
repeat until FINISHED = TRUE
{
  COD1 = computeCSCode(x1, y1) // calculeaza codul de 4 cifre pentru P(x1, y1)
  COD2 = computeCSCode(x2, y2) // calculeaza codul de 4 cifre pentru P(x2, y2)
  RESPINS = SimpleRejection(COD1, COD2) // testeaza cazul de respingere simpla
  if RESPINS = TRUE
    FINISHED = TRUE
  else
    {
      DISPLAY = SimpleAcceptance(COD1, COD2) // testeaza cazul de acceptare simpla
      if DISPLAY = TRUE
        FINISHED = true
      else
        {
          if(P(x1, y1) is inside the display area)
            invert(x1,y1,x2,y2,COD1,COD2) // daca P(x1,y1) este in interiorul zonei de afisare, inverseaza
                                           P(x1, y1) si P(x2, y2) impreuna cu codurile de 4 cifre

          if(COD1[1] = 1) and (y2 <> y1) // elimina segmentul aflat deasupra zonei de afisare
          {
            x1 = x1+(x2-x1)*(Ymax-y1)/(y2-y1)
            y1 = Ymax
          }
          elseif(COD1[2] = 1) and (y2 <> y1) // elimina segmentul aflat sub zona de afisare
          {
            x1 = x1+(x2-x1)*(Ymin-y1)/(y2-y1)
          }
        }
    }
}
```

```

        y1 = Ymin
    }
    elseif(COD1[3] = 1) and (x2 <> x1) // elimina segmentul aflat la dreapta zonei de afisare
    {
        y1 = y1 + (y2 - y1) * (Xmax - x1) / (x2 - x1)
        x1 = Xmax
    }
    elseif(COD1[4] = 1) and (x2 <> x1) // elimina segmentul aflat la stanga zonei de afisare
    {
        y1 = y1 + (y2 - y1) * (Xmin - x1) / (x2 - x1)
        x1 = Xmin
    }
}
}
}

```

5 Temă

- Implementați algoritmul Cyrus-Beck
- Implementați algoritmul Cohen-Sutherland
- Implementați (folosind SDL) o demonstrație interactivă a algoritmilor. Definiți, folosind mouse-ul, fereastra de decupare (un dreptunghi sau un poligon convex) și un segment de linie.

Laboratory work 6

1 Objectives

This laboratory highlights the Bresenham algorithms used for rendering some of the graphical primitives on a computer display. This paper begins by presenting some generic information about the algorithms and then exemplifies them for line and circle rasterization.

2 Theoretical background

The Bresenham algorithm for drawing lines onto a bi-dimensional space (like a computer display) is a fundamental method used in computer graphics discipline. The algorithm's efficiency makes it one of the most required methods for drawing continuous lines, circles or other graphical primitives. This process is called rasterization.

Each line, circle or other graphical primitives will be plotted pixel by pixel. Each pixel is described by a fixed (x, y) position in the bi-dimensional XOY space. The algorithm approximates the real line by computing each one of the line's pixel's position. Since the pixels are the smallest addressable screen elements in a display device, the algorithm approximation is good enough to "trick" the human eyes and to get the illusion of a real line. Figure 1a and Figure 1b shows the real line and the approximated line drawn over the pixel grid.

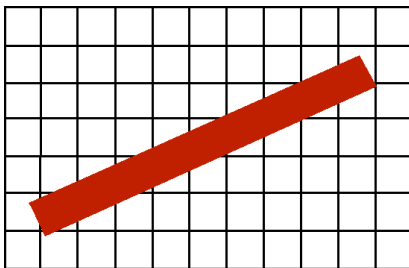


Figure 1a. Real line

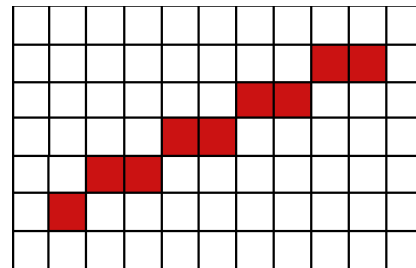


Figure 1b. Approximated line

Before moving on, it is worth to mention that both the line (1) and circle (2) can be mathematically described using the following equations:

$$y = m \cdot x + c \quad (1)$$

$$\text{with } m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$(x-a)^2 + (y-b)^2 = R^2 \quad (2)$$

where:

- m is the line's slope;
- $(x_1, x_2), (y_1, y_2)$ are the two endpoints of the line segment;
- (a, b) represents the coordinates of the circle's center;

2.1 Bresenham's algorithm for line

For simplicity we will take into account a line segment with the slope from 0 to 1. Suppose the two endpoints of the line are $A(x_1, y_1)$ and $B(x_2, y_2)$. At this point we have to choose an initial point to start the algorithm. We can choose this point $(P(x_i, y_i))$ to be either A or B. Based on the starting position, we have eight possible choices to draw the next pixel of the line. This is due to the fact that each pixel is surrounded by 8 adjacent pixels.

Our example will consider only the case where we have two choice alternatives for the next pixel position (in other words this example will work only for the first octant of the trigonometric circle). For example, for the current point P we have the following drawing possibilities: $T(x_{i+1}, y_i)$ or $S(x_{i+1}, y_{i+1})$.

The decision criterion (Figure 2) for Bresenham's algorithm is based on the distance between the current point, P, and the real line segment. So the closest point (T or S) to the line segment will be chosen.

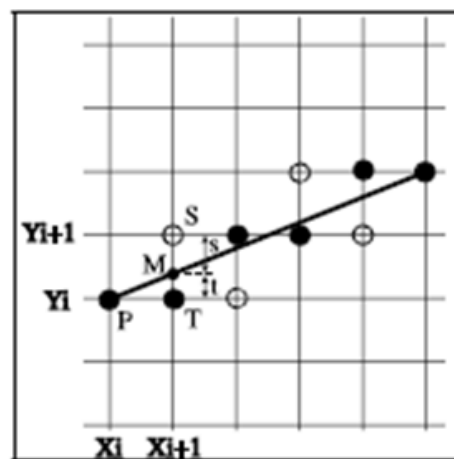


Figure 2. Decision criterion to choose the next line pixel that will plot on the screen display

The following paragraphs will describe the general steps of the Bresenham's algorithm in natural language rather than a programmatically one, because it is easier to understand.

- Let us assume that we have to draw a line segment with the endpoints represented by $A(x_1, y_1)$ and $B(x_2, y_2)$. We translate the line segment with $(-x_1, -y_1)$ to place it on the XOY system origin.
- Let $dx = x_2 - x_1$, $dy = y_2 - y_1$. The line that needs to be drawn can be described as $y = \frac{dx}{dy} x$.

- c. In this step we intend to compute the next line pixel, using the criterion mentioned above. From Figure 2, we can deduce that the closest point to the real line value is $T(x_{i+1}, y_i)$. Based

on this observation we could say that $M(x_{i+1}, x_{i+1} \cdot \frac{dy}{dx})$ (3).

$$\text{In other words } \begin{cases} t = y_m - y_i \\ s = y_{i+1} - y_m \end{cases} \Rightarrow t - s = 2 \cdot y_m - 2 \cdot y_i - 1 \quad (4).$$

Taking into account (3) and (4) we obtain $dx \cdot (t - s) = 2 \cdot dy \cdot x_{i+1} - 2 \cdot dx \cdot y_i - dx$. If the x coordinates of the line segment endpoints are in $x_1 < x_2$ relationship, then the sign of $t - s$ will coincide with the sign of $dx \cdot (t - s)$.

- d. We can obtain the following recurrence relationship: $d_{i+1} = d_i + 2 \cdot dy - 2 \cdot dx \cdot (y_i - y_{i+1})$ if we consider that $dx \cdot (t - s) = d_{i+1}$.

The initial value of $d_i = 2 \cdot dy - dx$ is obtained for $x_0 = 0$ and $y_0 = 0$. We can conclude that:

- If $d_i \geq 0 \Rightarrow (t - s) \geq 0$ and the closest point to the real line segment is $S(x_{i+1}, y_{i+1})$.

Based on this observations we find that d_i value can be computed as

$$d_{i+1} = d_i + 2(dy - dx).$$

- If $d_i < 0 \Rightarrow (t - s) < 0$ and the closest point to the real line segment is $T(x_{i+1}, y_i)$. Then the recurrence formula to compute d_i is $d_{i+1} = d_i + 2dy$.

The pseudo code for the Bresenham algorithm is described in the following paragraph, and it is based on the mathematical observations mentioned above.

```
//draw a line in the first octant
Algorithm Bresenham_line()
{
    //Initialize increments
    dx = abs(x2-x1);
    dy = abs(y2-y1);
    d = 2*dy-dx;
    inc1 = 2*dy;
    inc2 = 2*(dy-dx);

    //Set the starting point, end point and current point
    startX = x1;
    startY = y1;
    endX = x2;
    endY = y2;
    currentX = x1;
    currentY = y1;

    //Draw each pixel of the line
    while (currentX < endX) {
```

```

//Draw the current pixel
DrawPixel(currentX, currentY);
increment currentX;

    if (d < 0) then {
        increment d using inc1;
    }
    else {
        increment currentY;
        increment d using inc2;
    }
}
}

```

2.2 Bresenham's algorithm for circle

Let's say we want to scan-convert a circle centered at $(0, 0)$ with an integer radius R (Figure 3). We'll see that the ideas we previously used for line scan-conversion can also be used for this task. First of all, notice that the interior of the circle is characterized by the inequality $D(x, y) : x^2 + y^2 - R^2 < 0$.

We'll use $D(x, y)$ to derive our decision variable.

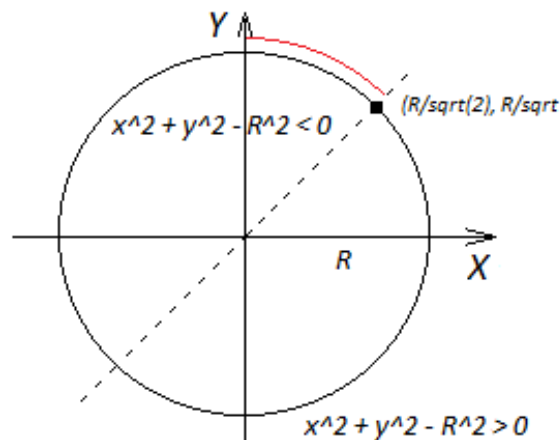


Figure 3. A circle and the description of its interior and exterior as two quadratic inequalities

Following the same approach as for the line segment representation, we'll first present the Bresenham's algorithm for the circle in natural language, describing for each step the general ideas behind it.

- a. First, let's think how to plot pixels close to the 1/8 of the circle marked red in Figure 3. The range of the x coordinate for such pixels is from 0 to $R\sqrt{2}$. We'll go over vertical scanlines through the centers of the pixels and, for each such scanline, compute the pixel on that line

which is the closest to the scanline-circle intersection point (black dots in Figure 4). All such pixels will be plotted by our procedure.

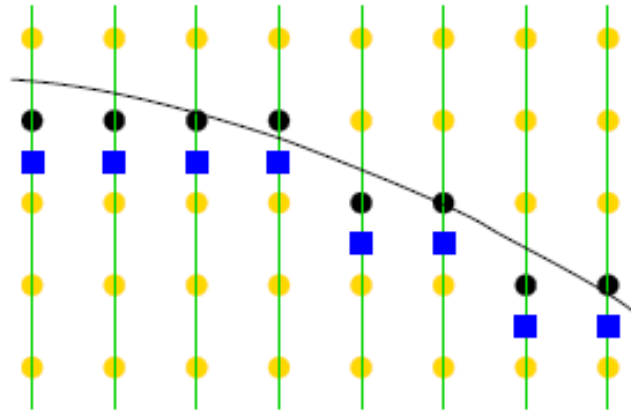


Figure 4. A circle and the description of its interior and exterior as two quadratic inequalities

- b. Notice that each time we move to the next scanline, the y-coordinate of the plotted point either stays the same or decreases by 1 (the slope of the circle is between -1 and 0). To decide what needs to be done, we'll use the decision variable, which will be the value of $D(x, y)$ evaluated at the blue square (e.g. the midpoint between the plotted pixel and the pixel immediately below).
- c. The first pixel plotted is $(0, R)$ and therefore the initial value of the decision variable should be

$$D(0, R-0.5) = (R-0.5)^2 - R^2 = 0.25 - R$$

The y variable, holding the second coordinates of the plotted pixels, will be initialized to R . Let's think now at what happens after a point (x, y) is plotted. First, we'll pretend that we need to move the plotted point to the right (no change in y) and check if this keeps the decision variable negative (we don't want any blue squares outside the circle). If (x, y) is the last plotted point, the decision variable is $D(x, y-0.5)$. After we move to the right, it becomes $D(x+1, y-0.5)$. Simple arithmetic shows that it increases by $D(x+1, y-0.5) - D(x, y-0.5) = 2x+1$. If this increase makes it positive, we'd better move down by 1 pixel. This puts the blue square at $(x+1, y-1.5)$ and means that we need to increase the decision value by the previous $2x+1$ plus $D(x+1, y-1.5) - D(x+1, y-0.5) = 2-2y$.

- d. Clearly, to make the decision variable integer, we need to scale it by a factor of 4. Eight-way symmetry is used to go from 1/8-th of the circle to the full circle.

The pseudo code for the Bresenham's algorithm for circle is described below, based on the earlier made observations.

```

Algorithm Bresenham_circle ()
{
    currentY = R;
    d = 1/4 - R;

    //Go only one eighth of a circle
    for currentX = 0 to ceil(R/sqrt(2)) do {

        plot_points(currentX,currentY);
        increment the decision variable by 2*currentX + 1;

        if (d > 0) then
        {
            increment the decision variable by 2 - 2*currentY;
            decrement currentY;
        }
    }
}

```

You can find the **plot_points** function definition below:

```

Function plot_points (x, y)
{
    DrawPixel (x,y);
    DrawPixel (x,-y);
    DrawPixel (-x,y);
    DrawPixel (-x,-y);
    DrawPixel (y,x);
    DrawPixel (-y,x);
    DrawPixel (y,-x);
    DrawPixel (-y,-x);
}

```

3 Geometry rendering using SDL's API

The SDL 2.0 library provides a hardware accelerated rendering API for basic shapes such as rectangles, lines or points.

In order to use hardware accelerated rendering, we have to create a new **SDL_Renderer** for our SDL window.

```

//The window renderer
SDL_Renderer* renderer = NULL;
...
//Create renderer for window
renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);

```

To select a rendering color and to clear the window we can use **SDL_SetRenderDrawColor** and **SDL_RenderClear**.

```
//Clear screen
SDL_SetRenderDrawColor(renderer, 0xFF, 0xFF, 0xFF, 0xFF);
SDL_RenderClear(renderer);
```

Whenever a primitive is drawn, its color is the currently selected rendering color. To render points (pixels) we can use **SDL_RenderDrawPoint**.

```
//Draw current point
SDL_RenderDrawPoint(renderer, tmpCurrentX, tmpCurrentY);
```

4 Assignment

- Explore the implementation of Bresenham's algorithm for drawing lines provided in the laboratory's resources folder.
- Extend Bresenham's algorithm implementation for drawing lines to work in all octants of the trigonometric circle. The algorithm presented in this document, as well as the sample code cover only the first octant. **Note: Be careful when implementing Bresenham's algorithm. The SDL system of coordinates is different from the Cartesian system of coordinates.**
- Implement the function to draw circles using Bresenham's algorithm in the provided sample code.

Laborator 9

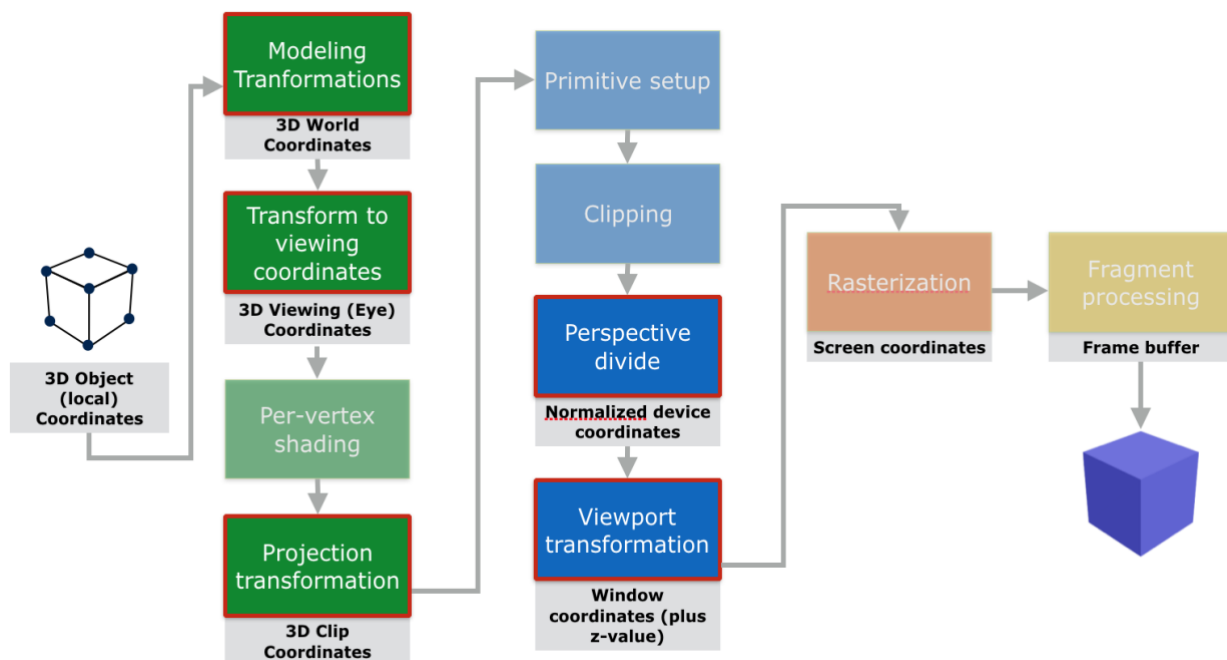
1 Obiective

Acest laborator prezintă transformările de vizualizare utilizate pentru a mapa locațiile 3D (specificate prin coordonatele x , y și z) la coordonatele 2D (specificate prin coordonatele pixelilor) ale ecranului.

2 Transformări de vizualizare

Pentru a mapa coordonatele 3D la coordonate 2D, folosim o secvență de trei transformări:

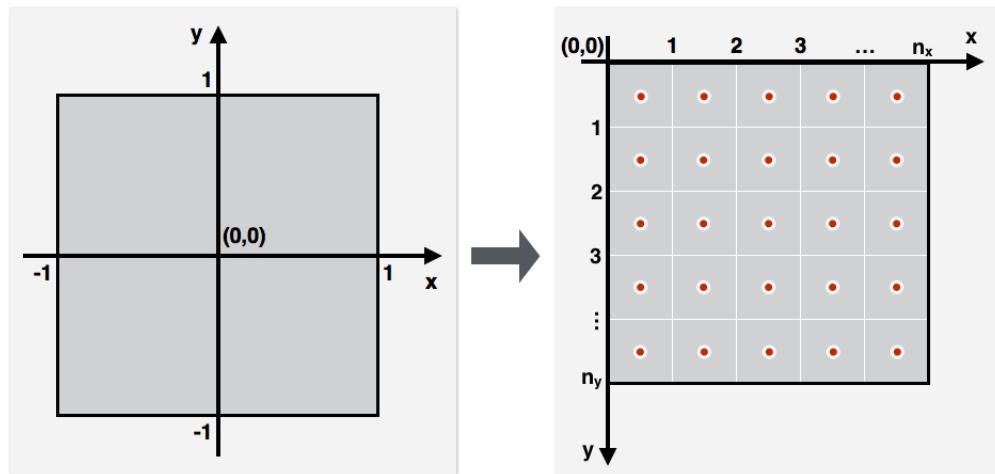
- **Transformări ale camerei**
 - Folosite pentru a plasa camera în origine și pentru a re poziționa toate celelalte obiecte în raport cu noua poziție a camerei
 - Această transformare depinde numai de poziția și orientarea camerei
- **Transformări de proiecție**
 - Utilizat pentru a proiecta puncte din spațiul de coordonate ale camerei
 - După transformare toate punctele vizibile vor fi în intervalul $[-1, 1]$
 - Această transformare depinde numai de tipul de proiecție (perspectivă sau ortogonală)
- **Transformări de viewport**
 - Folosite pentru a mapa dreptunghiul unitate din spațiul imaginii la dreptunghiul dorit în coordonate pixeli
 - Această transformare depinde numai de mărimea și poziția imaginii rezultate



Pe parcursul acestor transformări schimbăm sistemele de coordonate în care sunt specificate obiectele. Transformarea camerei schimbă coordonatele din **spațiul global** (world space) în **spațiul camerei**. Transformarea de proiecție mută puncte din **spațiul camerei** în **volumul de vizualizare canonic** (aici decuparea este efectuată mai eficient). Ultima transformare, transformarea ferestrei de vizualizare (viewport) mapează **volumul de vizualizare canonic** la **spațiul ecran** (screen space).

3 Transformarea ferestrei de vizualizare (Viewport)

Această transformare este utilizată pentru a mapa puncte din volumul de vizualizare canonic (unde valorile sunt în intervalul $[-1, 1]$) la spațiul ecran (definit de lățimea și înălțimea imaginii rezultate). Aceasta se compune din mai multe transformări, inclusiv translație, scalare și reflecție. Originea imaginii este considerată colțul din stânga sus. Pentru alte specificații ale spațiului imaginii, transformările ar putea fi diferite. Centrul pixelilor este considerat a fi la valoarea întregă plus 0,5 (ambele pe axa x și y).



$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{S} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & 0 \\ 0 & \frac{n_y}{2} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

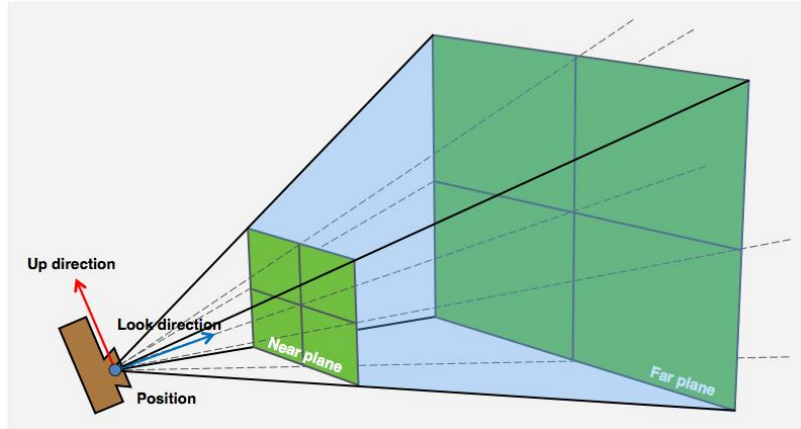
$$\mathbf{M}_{vp} = \mathbf{SMT} = \begin{bmatrix} \frac{n_x}{2} & 0 & 0 & \frac{n_x}{2} \\ 0 & -\frac{n_y}{2} & 0 & \frac{n_y}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Pentru a afișa imaginea pornind de la o altă locație decât punctul (0, 0) trebuie să adăugăm o transformare de translație adițională matricii $\mathbf{M}_{vp}$.

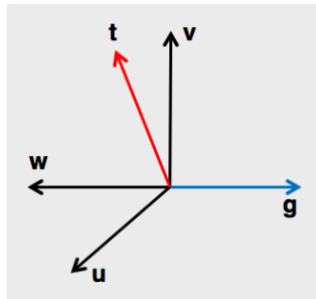
$$\mathbf{M}_{vp} = \mathbf{T}(\text{startx}, \text{starty}) * \mathbf{S} * \mathbf{M} * \mathbf{T}$$

4 Transformarea camerei

O cameră virtuală este definită de un set de atribute și parametri, cum ar fi: Poziția camerei, Orientare, Câmpul de vizualizare, Tipul de proiecție (perspectivă sau paralelă), Adâncimea câmpului, Distanța focală, Înclinarea și decalarea lentilei camerei relative la corpul camerei. Pentru un model de bază al camerei trebuie să specificăm cel puțin poziția acesteia, orientarea și câmpul vizual.



Camera este specificată în coordonate globale prin următorii parametri: poziția $\mathbf{e}$, direcția de vizitare $\mathbf{g}$, vectorul de vizualizare $\mathbf{t}$. Din acești parametri trebuie să definim un sistem de coordonate $\mathbf{uvw}$.



$$\begin{aligned}\mathbf{w} &= -\frac{\mathbf{g}}{\|\mathbf{g}\|} \\ \mathbf{u} &= \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|} \\ \mathbf{v} &= \mathbf{w} \times \mathbf{u}\end{aligned}$$

Trebuie să aliniem cele două sisteme de coordonate diferite (axele u, v, w cu axele x, y, z) folosind următoarea matrice de transformare:

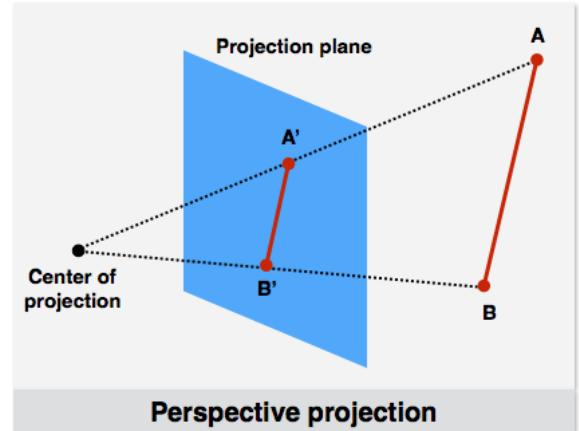
$$\mathbf{M}_{cam} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5 Transformarea de proiecție

Două tipuri de proiecții sunt prezentate în acest document, proiecția perspectivă și cea paralelă.

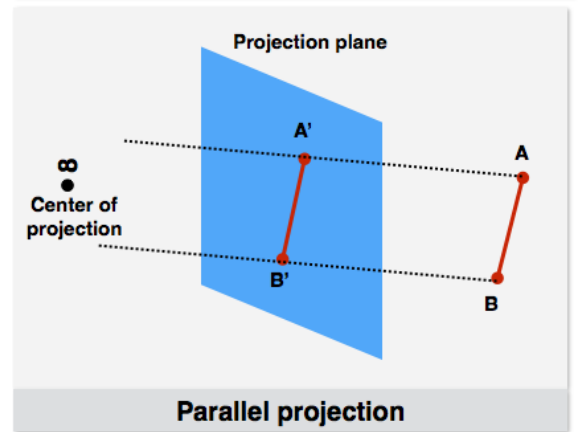
Proiecția perspectivă:

- Razele de proiecție converg în centrul proiecției (locația observatorului)
- Obiectele apar mai mici cu creșterea distanței față de centrul proiecției (ochiul observatorului)
- Liniile paralele cu planul de proiecție rămân paralele
- Liniile care nu sunt paralele cu planul de proiecție converg către un singur punct (punct de dispariție)



Proiecția paralelă:

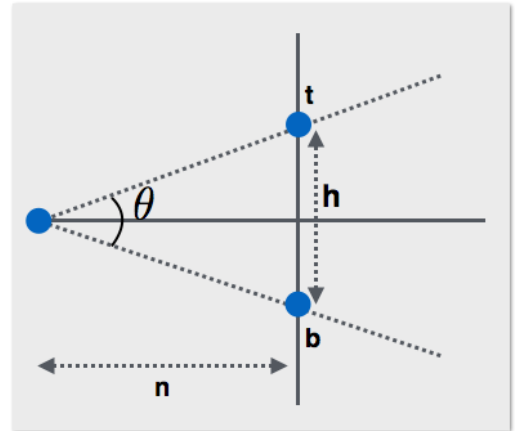
- Raze de proiecție paralele
- Punctul de convergență la infinit
- Poziția observatorului la infinit
- Liniile paralele rămân paralele



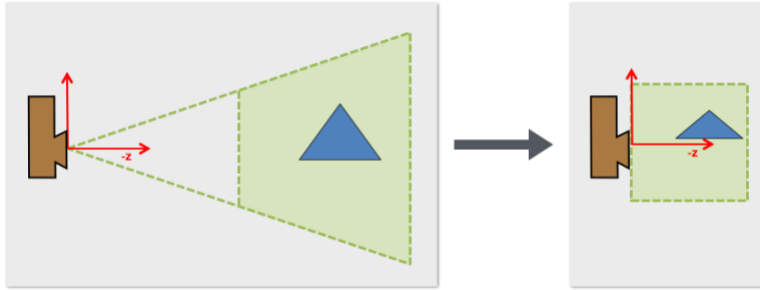
$$M_{per} = \begin{bmatrix} -\frac{1}{\text{aspect} \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & -\frac{1}{\tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\tan \frac{\theta}{2} = \frac{h}{2|n|}$$

$$\text{aspect} = \frac{\text{width}}{\text{height}} = \frac{r-l}{t-b}$$



Înainte de a afișa nodurile, trebuie să efectuăm o operație denumită **diviziune de perspectivă**.



Planele de apropiere și de depărtare sunt definite în coordonatele camerei. Deoarece camera este localizată în origine și este orientată în direcția negativă z, valorile de apropiere și de depărtare ar trebui să fie negative, cu n(near) > f(far).

$$P = M_p \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} xd \\ yd \\ zd \\ z \end{bmatrix} \xrightarrow{\text{After perspective divide}} \begin{bmatrix} \frac{d}{z}x \\ \frac{d}{z}y \\ d \\ 1 \end{bmatrix}$$

6 Combinarea transformărilor

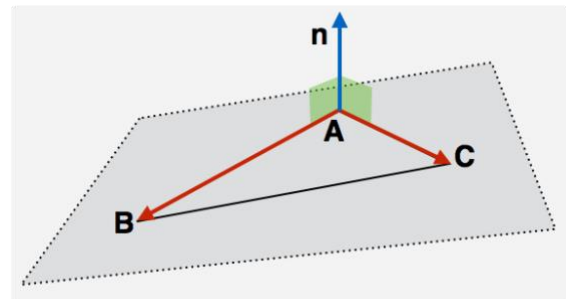
```
construim Mvp
construim Mper
construim Mcam
M = Mvp * Mper * Mcam
for each segment de linie(ai,bi) do
    p = M * ai
    q = M * bi
    drawline(xp/wp,yp/wp,xq/wq,yq/wq)
```

7 Calculul și afișarea vectorului normală al unui triunghi

Pentru a calcula vectorul normală, putem folosi următoarea formulă, bazată pe produsul vectorial dintre două vectori. Ultimul pas este de a normaliza vectorul rezultat pentru a obține un vector normală.

$$\mathbf{n} = (B - A) \times (C - A)$$

Pseudocodul pentru afișarea vectorului normală:



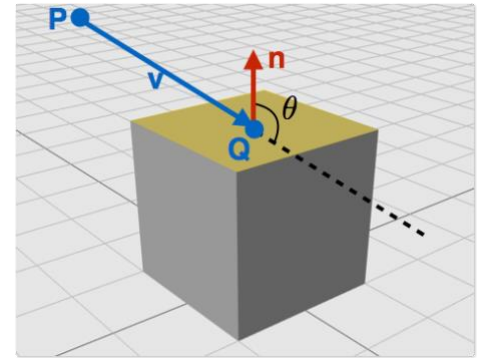
```
displayNormalVector()
    centerPoint = calculati punctul central al triunghiului
    normalVector = calculati vectorul normala
    secondPoint = centerPoint + normalVector * offset
    drawLine(centerPoint, secondPoint)
```

8 Eliminarea fețelor orientate invers

În funcție de poziția relativă a camerei față de triunghiurile obiectului, putem identifica triunghiurile vizibile.

Poziția relativă a unui punct **P** (poziția camerei) față de un plan (triunghi):

- $\theta > 90^\circ$ atunci P se află în fața planului
- $\theta = 90^\circ$ atunci P se află pe plan
- $\theta < 90^\circ$ atunci P se află în spatele planului



Unghiul poate fi calculat folosind produsul scalar dintre cei doi vectori, **v** și **n**.

9 Decuparea în coordonate omogene

Putem decupa punctele pe baza următoarelor plane de decupare:

- $P.w \leq P.x \leq P.w$
- $P.w \leq P.y \leq P.w$
- $P.w \leq P.z \leq P.w$

10 Temă

Descărcați codul sursă și implementați următoarele metode (din projection.h):

- `bool clipPointInHomogeneousCoordinate(const egc::vec4 &vertex)`
- `bool clipTriangleInHomogeneousCoordinates(const std::vector<egc::vec4> &triangle)`
- `egc::vec3 findNormalVectorToTriangle(const std::vector<egc::vec4> &triangle)`
- `egc::vec4 findCenterPointOfTriangle(const std::vector<egc::vec4> &triangle)`
- `bool isTriangleVisible(const std::vector<egc::vec4> &triangle, const egc::vec3 &normalVector)`
- `void displayNormalVectors(egc::vec3 &normalVector, egc::vec4 &triangleCenter)`
- `mat4 defineViewTransformMatrix(int startX, int startY, int width, int height)`
- `mat4 defineCameraMatrix(Camera myCamera)`
- `mat4 definePerspectiveProjectionMatrix(float fov, float aspect, float zNear, float zFar)`
- `void perspectiveDivide(vec4 &inputVertex)`

Laborator 10

1 Obiective

Această lucrare de laborator prezintă noțiuni cheie despre curbele Bezier.

2 Fundament teoretic

2.1 Ecuția parametrică a liniei

Un segment de linie definit de punctele P_0 și P_1 poate fi complet specificat utilizând următoarea ecuație:

$$P(t) = (1 - t) * P_0 + t * P_1$$

Această ecuație descrie un punct dat pe segmentul de linie ca o combinație liniară a punctelor finale. În cadrul ecuației, t este parametrul și variază în intervalul $[0, 1]$. Alegând o valoare pentru t în intervalul $[0, 1]$, puteți calcula coordonatele oricărui punct de pe acel segment de linie.

Coordonatele pot fi calculate prin aplicarea formulei de mai sus atât pe dimensiunea X, cât și pe Y - Figura 1.

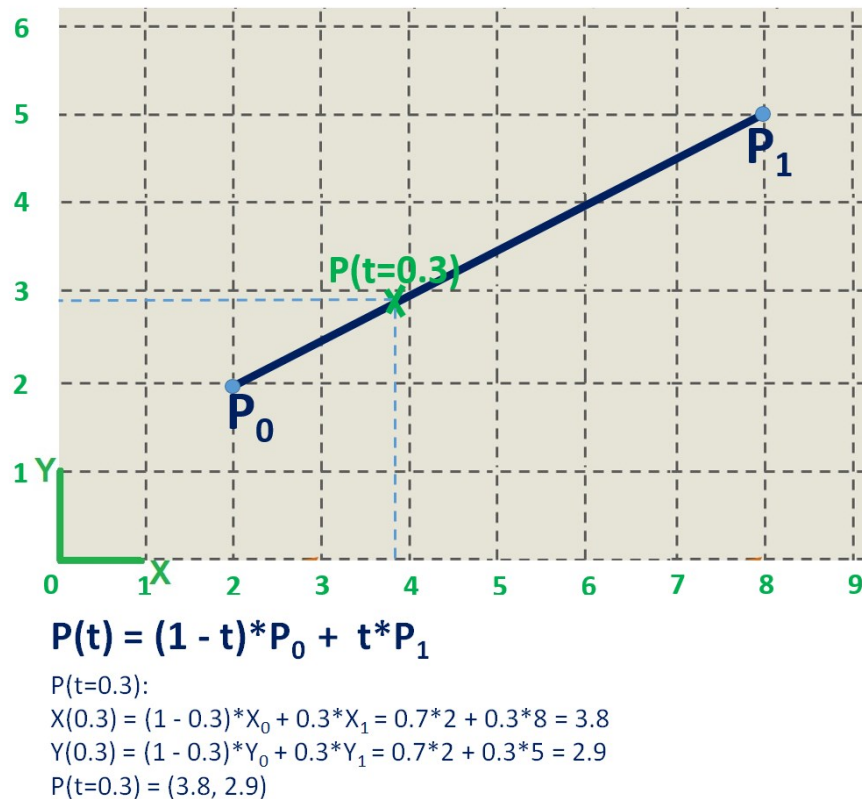


Figure 1 - Ecuția parametrică a liniei - exemplu aplicat

2.2 Curbe Bezier

O curbă Bezier este o curbă parametrică, utilizată în grafica computerizată și alte câmpuri conexe, pentru a modela curbe netede care pot fi scalate la infinit - Figura 2. Curbele Bezier sunt definite cu ajutorul unei serii de puncte - numite puncte de control - care pot influența forma lor generală.

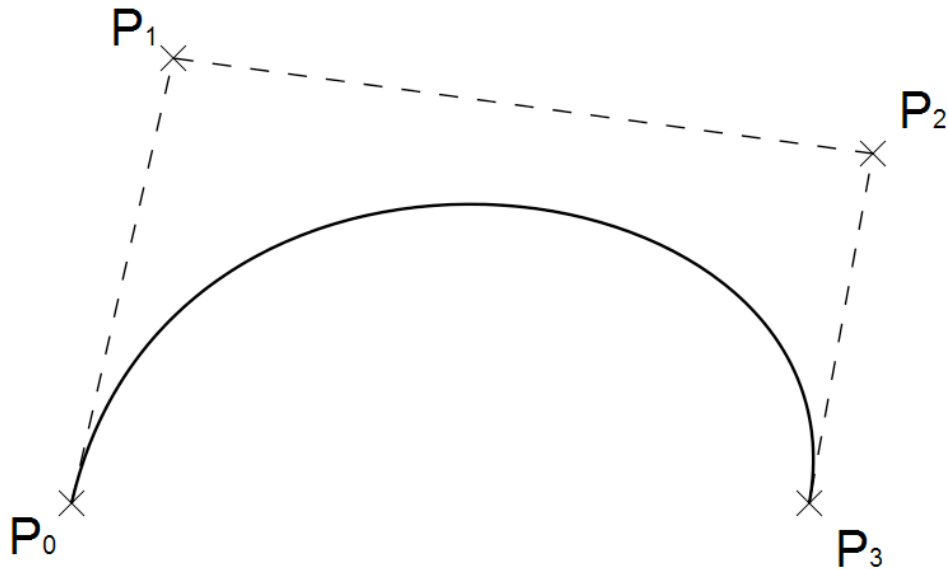


Figure 2 - Example of a cubic Bezier curve

Mulțimea punctelor de pe curba Bezier este definită ca o combinație polinomială a punctelor de control:

$$B(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i P_i$$

De exemplu, o curbă Bezier de gradul I (definită folosind două puncte de control) este descrisă prin următoarea ecuație:

$$B_{P_0, P_1}(t) = (1-t)*P_0 + t*P_1$$

Se poate observa asemănarea cu ecuația parametrică a liniei. Aceasta înseamnă că vă puteți gândi la o linie ca la o curbă Bezier de gradul I.

O curbă Bezier de gradul doi (numită și pătratică) este definită de următoarea ecuație:

$$B_{P_0, P_1, P_2}(t) = (1-t)^2*P_0 + 2(1-t)t*P_1 + t^2*P_2$$

Se pot defini curbe Bezier folosind oricât de multe puncte de control, obținându-se rezultate de complexitate sporită.

2.3 Formula recursivă

Probabil că este mai ușor să înțelegeți și să aplicați formula Bezier în forma sa recursivă:

$$B_{P_0, P_1, \dots, P_n}(t) = (1-t) * B_{P_0, P_1, \dots, P_{n-1}}(t) + t * B_{P_1, P_2, \dots, P_n}(t)$$

Conform acestei formule, calculul coordonatelor unui punct (cu parametrul t) pe o curbă Bezier definită de n puncte de control se poate face prin aplicarea ecuației parametrice liniare pe rezultatele curbelor Bezier de ordin inferior, definite de $n-1$ puncte de control.

Curba Bezier de ordinul cel mai mic este cea definită de două puncte de control, așa cum se arată în pagina anterioară. Aceasta poate fi calculată prin aplicarea ecuației parametrice liniare pe punctele sale de control:

$$B_{P_0, P_1}(t) = (1-t) * P_0 + t * P_1$$

Figura 2 exemplifică metoda recursive de calcul a punctelor pe o curbă Bezier de gradul doi. În acest exemplu, se calculează două puncte intermediare pentru curbă: $t = 0,3$ și $t = 0,6$.

Observați aplicarea ecuației parametrice asupra rezultatelor curbelor Bezier de ordin inferior - de-a lungul liniilor verzi și albastre.

De asemenea, observați că liniile albastre și verzi au capetele definite prin aplicarea ecuației parametrice pe liniile P_0P_1 și P_1P_2 .

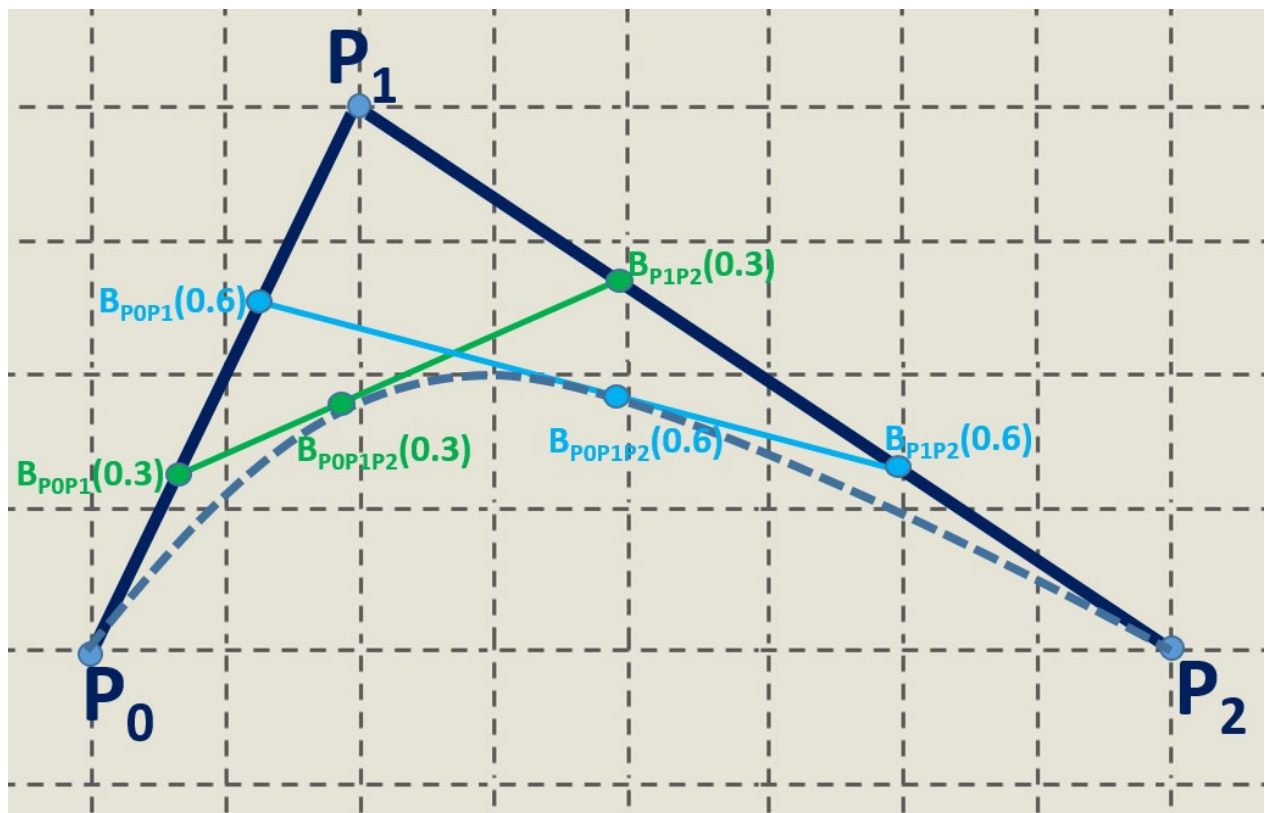


Figure 2 – Curba Bezier cu 3 puncte de control - calculul recursiv al punctelor

3 Implementare

1. Implementați o metodă care colorează pixelii dintre punctele P_0 (400, 70) și P_1 (600, 400). Utilizați ecuația parametrică a liniei pentru a găsi pixelii care se află pe acea linie.
 - Modificați incrementul parametrului t pentru a maximiza numărul de puncte obținute.
2. Exemplificați trasarea unei curbe pătratice Bezier. Implementați o metodă care colorează pixelii de-a lungul curbei. Folosiți formula recursivă Bezier pentru a găsi punctele.
 - Modificați incrementul parametrului t pentru a crește / micșora numărul de puncte obținute.
3. Implementați funcționalitatea pentru trasarea curbelor Bezier de ordin superior definind punctele de control folosind acțiuni de click.
 - Ar trebui să calculați un număr mic de puncte de-a lungul curbei (pași mai mari pentru t) pentru a economisi puterea de calcul.
 - Încercați să trasați linii între punctele găsite de-a lungul curbei, pentru a crea o aproximare vizuală a acesteia.

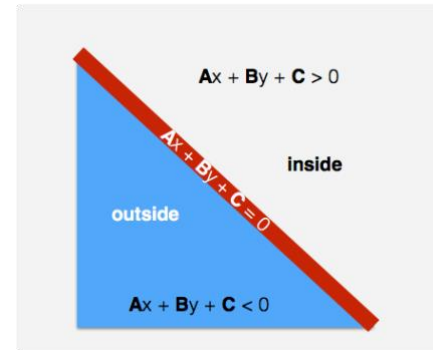
Laborator 10

1 Obiective

Acest laborator prezintă operația de rasterizare a triunghiurilor folosind coordonate baricentrice.

2 Definirea triunghiurilor

Definim fiecare triunghi prin muchii și determinăm ecuațiile acestor muchii astfel încât vectorul perpendicular pe fiecare muchie să fie îndreptat spre interiorul triunghiului. Începem de la forma generală implicită a unei linii (în 2D) $Ax + By + C = 0$. Forma implicită a unei linii care trece prin punctele $A(x_a, y_a)$ și $B(x_b, y_b)$ este: $(y_a - y_b)x + (x_b - x_a)y + x_a y_b - x_b y_a = 0$.

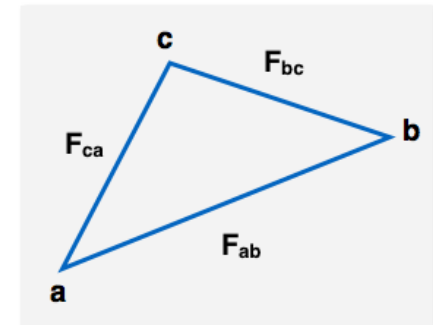


Pentru un triunghi definit de vârfurile $a(x_a, y_a)$, $b(x_b, y_b)$ și $c(x_c, y_c)$ avem următoarele ecuații pentru muchiile care îl definesc:

$$F_{ab} = (y_a - y_b)x + (x_b - x_a)y + x_a y_b - x_b y_a = 0$$

$$F_{bc} = (y_b - y_c)x + (x_c - x_b)y + x_b y_c - x_c y_b = 0$$

$$F_{ca} = (y_c - y_a)x + (x_a - x_c)y + x_c y_a - x_a y_c = 0$$



2.1 Coordonate baricentrice

Oricare punct p este o combinație liniară a punctelor P , Q , și R :

$$\begin{aligned} p &= P + \beta(Q - P) + \gamma(R - P) \\ &= (1 - \beta - \gamma)P + \beta Q + \gamma R \\ &= \alpha P + \beta Q + \gamma R \end{aligned}$$

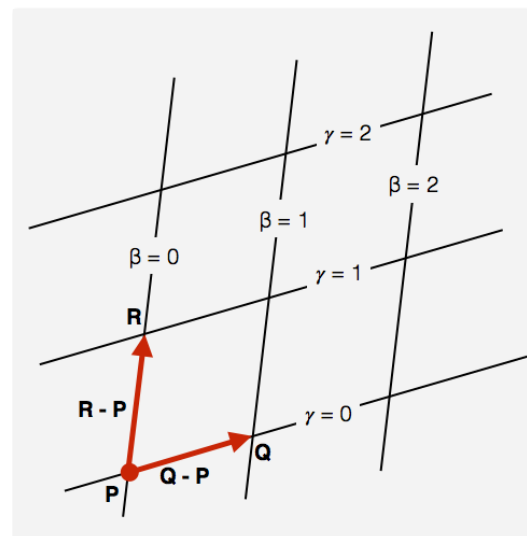
Pentru un triunghi:

$$\beta + \gamma \leq 1$$

$$\beta \geq 0$$

$$\gamma \geq 0$$

α, β, γ sunt denumite **coordonațe baricentrice**.



Coordonatele baricentrice descriu un punct p ca o combinație afină a vârfurilor triunghiului:

$$p = \alpha P + \beta Q + \gamma R, \text{ where } \alpha + \beta + \gamma = 1$$

Pentru oricare punct p din interiorul unui triunghi definit de vârfurile a, b, și c:

$$0 < \alpha < 1$$

$$0 < \beta < 1$$

$$0 < \gamma < 1$$

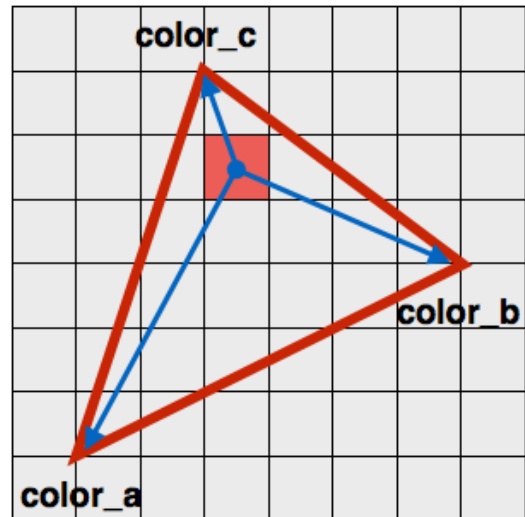
Un punct este pe muchie dacă una dintre coordonatele baricentrice este egală cu 0, și un punct este pe un vârf al triunghiului dacă două coordonate baricentrice sunt egale cu 0.

Pentru un triunghi coordonatele baricentrice ale unui punct p sunt:

$$\alpha = \frac{F_{bc}(x, y)}{F_{bc}(x_a, y_a)}$$

$$\beta = \frac{F_{ac}(x, y)}{F_{ac}(x_b, y_b)}$$

$$\gamma = 1 - \alpha - \beta$$



2.2 Pseudocod rasterizare triunghi

```
void triangleRasterization(vertices v[3]){
    bbox b = findBoundingBox(v);
    foreach pixel(x, y) in b
    {
        compute alpha, beta, gamma;
        if(0 < alpha < 1 and
           0 < beta < 1 and
           0 < gamma < 1)
        {
            color = color_a * alpha + color_b * beta + color_c * gamma;
            drawPixel(x, y) with color;
        }
    }
}
```

```
}
```

3 Temă

- Extindeți implementarea din laboratorul precedent pentru a adăuga ca și funcționalitate afișarea obiectelor 3D folosind algoritmul de rasterizare a triunghiurilor.

Laborator 11

1 Obiective

Această lucrare de laborator evidențiază o tehnică pentru îndepărtarea suprafețelor ascunse utilizând algoritmul z-buffer.

2 Fundament teoretic

Determinarea suprafețelor vizibile și ascunse este o problemă fundamentală în grafica asistată de calculator. De obicei, este extrem de ineficient să procesăm obiecte care nu vor apărea niciodată în imaginea finală.

Am văzut deja o tehnică pentru îndepărtarea suprafețelor ascunse în lucrarea de laborator cu numărul 9, și anume *Back Face Culling*, care îndepărtează primitivele aflate în partea din spate a obiectelor opace din pipeline-ul grafic. Algoritmul z-buffer elimină primitive care ar apărea în spatele altor obiecte opace.

Z-buffer-ul, cunoscut și ca buffer de adâncime, este o matrice 2D având aceleași dimensiuni ca framebuffer-ul (imaginea raster finală) care păstrează, pentru fiecare pixel, distanța de la centrul de proiecție la suprafața care a dat culoarea finală a pixelului. Figura 1 prezintă un exemplu al conținutului framebuffer-ului și al z-buffer-ului atunci când se trasează un triunghi roșu paralel cu axa Z, situat la o adâncime de $z = -5$.

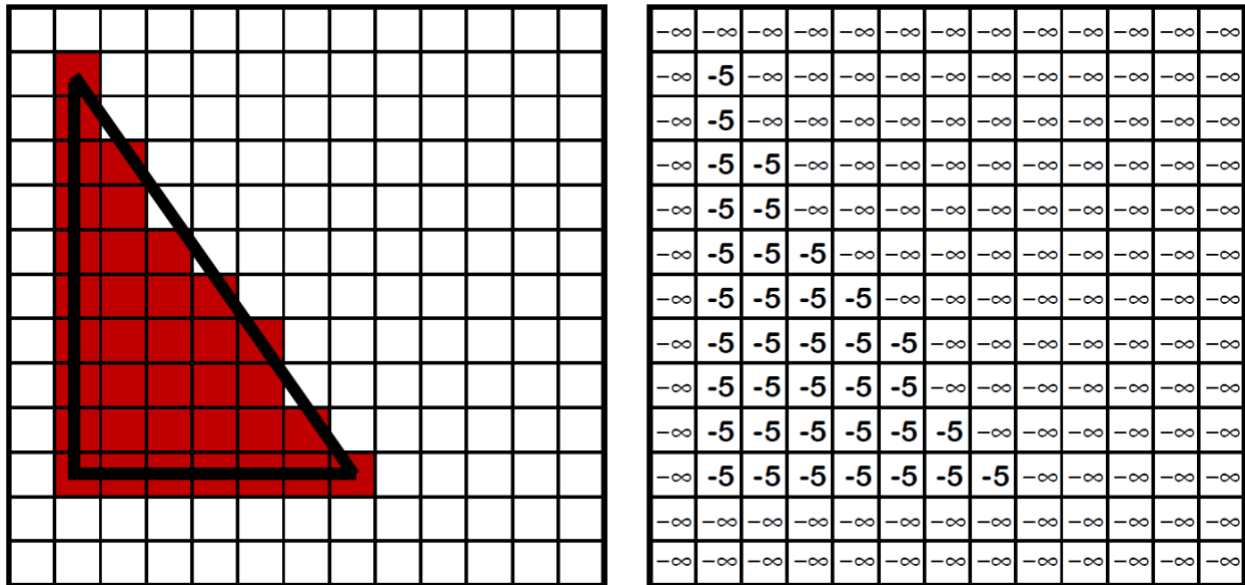


Figure 1 – Frame-buffer și z-buffer pentru desenarea unui triunghi roșu cu $z = -5$

Când o nouă primitivă este rasterizată, adâncimea fiecăruia dintre pixelii candidați este comparată cu valoarea de adâncime existentă în z-buffer. Pixelul final este actualizat numai dacă valoarea pixelului candidat este mai aproape de centrul de proiecție decât valoarea aflată deja în buffer. Dacă pixelul este actualizat, valoarea adâncimii existente în z-buffer este înlocuită cu adâncimea noului pixel.

De exemplu, la trasarea unui triunghi galben paralel cu axa Z și situat la o adâncime de $z = -7$, dacă nu există primitive procesate anterior, framebuffer-ul rezultat și z-buffer-ul sunt cele ilustrate în Figura 2.

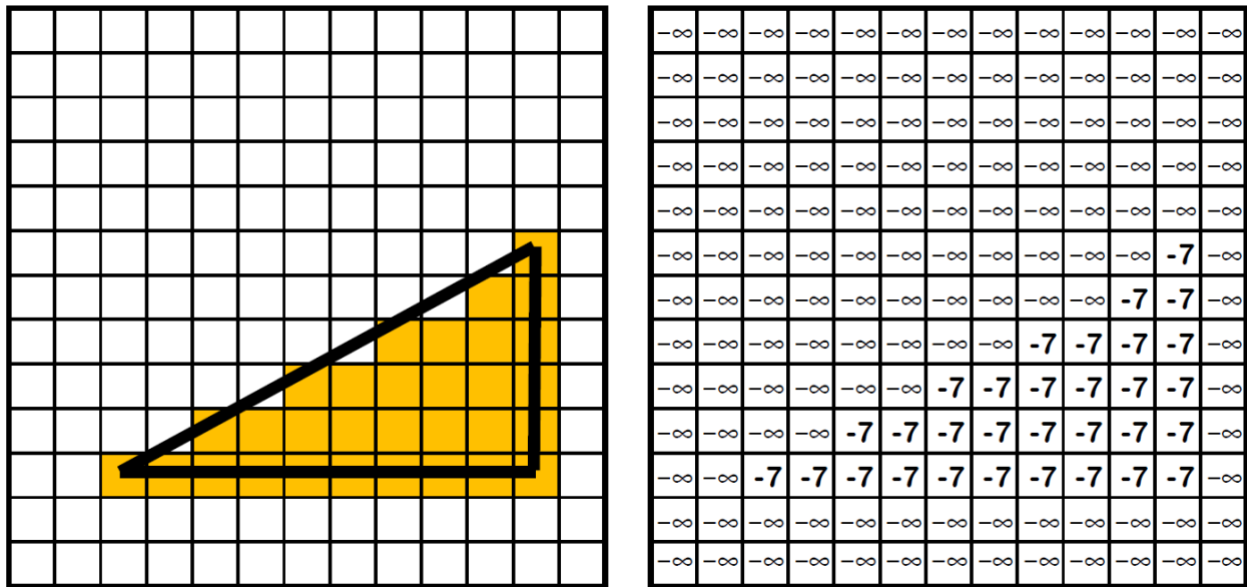


Figure 2 - Frame-buffer și z-buffer pentru desenarea unui triunghi galben $z = -7$

Dacă triunghiul galben este rasterizat după triunghiul roșu, atunci conținutul framebuffer-ului și al z-buffer-ului sunt ilustrate în Figure 3.

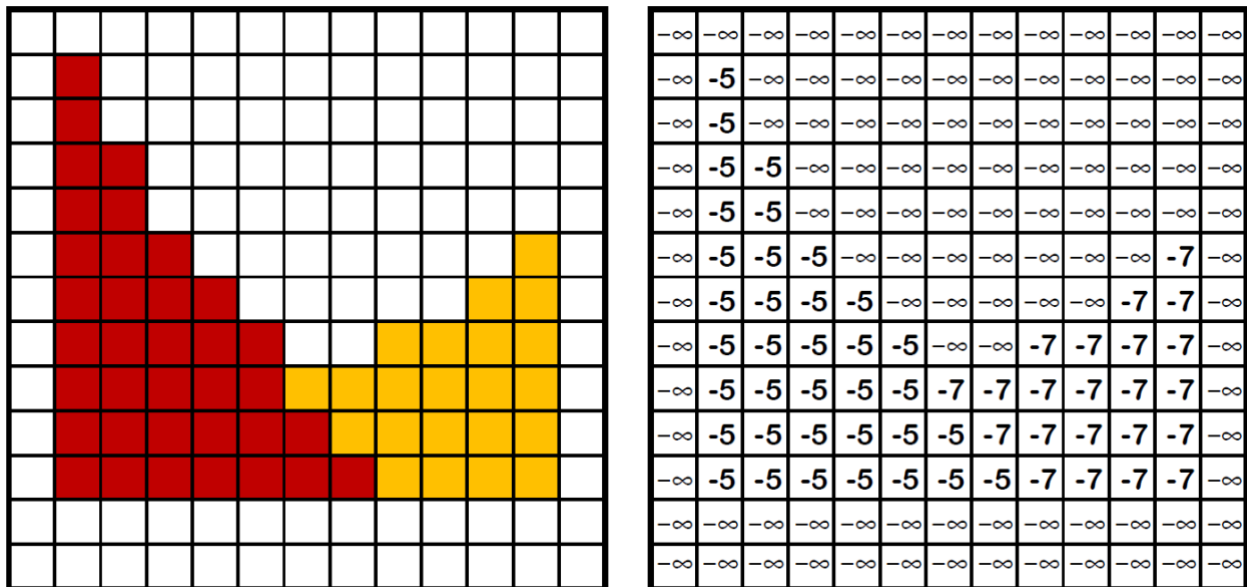


Figure 3 - Frame-buffer și z-buffer pentru desenarea unui triunghi roșu cu $z = -5$ și a unui triunghi galben $z = -7$

Cea de-a doua primitivă procesată actualizează z-buffer-ul doar pe pozițiile pixelilor pentru care adâncimea candidatului este mai aproape de centrul proiecției decât adâncimea existentă. Celelalte poziții rămân neschimbate.

3 Colorarea iepurelui pe bază de adâncime

Pentru a obține o mai bună înțelegere a importanței îndepărtării suprafețelor ascunse, vom folosi o schemă de colorare bazată pe adâncime pentru iepurele din laboratorul 9. Algoritmul va calcula fiecare culoare de vârf pe baza coordonatei sale inițiale Z (în spațiul obiect) folosind formula:

$$Color_{RGB} = (depthCoeff, depthCoeff, depthCoeff),$$

unde

$$depthCoeff = 255 * (1 - \frac{vertex_z - min_z}{max_z - min_z})$$

$Vertex_z$ este coordonata Z a vârfului current, max_z and min_z sunt valorile Z maxime și minime dintre toate vârfurile iepurelui.

Rezultatul ar trebui să semene cu imaginea din Figure 4.

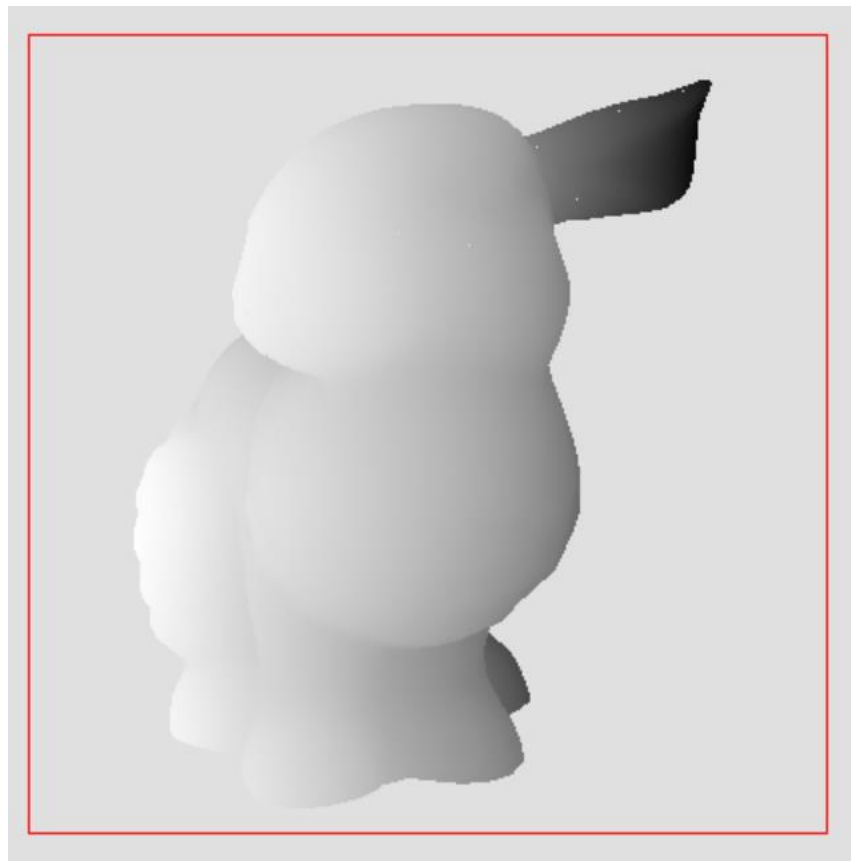


Figure 4 – Colorare pe bază de adâncime

4 Temă

- Aplicați schema de colorare pe bază de adâncime pentru iepurele rasterizat în laboratorul 10.
- Extindeți funcția de rasterizare a triunghiului din laboratorul 9 pentru a include un algoritm de eliminare a suprafețelor ascunse bazat pe z-buffer.

- Comutați interactiv între rasterizarea cu/fără z-buffer, folosind tastatura. Pentru a vedea impactul complet al algoritmului, asigurați-vă că opriți Back Face Culling.